Exploring Guarded Interaction Trees

Cheng (Charles) Peng

June 15, 2026

1 Overview

During the semester of spring 2026, I conducted a project work titled *Exploring Guarded Interaction Trees* under supervision of Prof. Lars Birkedal. The primary objective was to understand how to develop denotational semantics for effectful programming languages in guarded interaction trees (GItrees). The project was carried out in four phases:

1. In the preparation phase, I familiarized myself with writing formal proof using the Iris higher-order separation logic framework within the Rocq proof assistant.
2. Subsequently, I studied the two research papers on guarded interaction trees [Ste+25; FTB24] to learn the general theory of GItrees. I learned using GItrees to interpret programs involving higher-order effects.
3. Afterwards, I proposed and completed a case study on interpreting pseudo-randomness as an effect in GItrees. I designed a simple programming language with pseudo-random number generator functionality and developed a denotational semantics based on GItrees for proving safety and contextual equivalence. The results have been mechanized in Rocq as an extension to the GItree Rocq formalization[1].
4. In the final stage, I experimented with modeling and reasoning about interactions between controller programs and physical devices, which is common in Internet of Things (IoT) applications.

The current document summarizes my learning outcomes and preliminary exploration results. It is written for a three-fold purpose: (1) to provide a summary of my work for examination; (2) to articulate initial observations and insights for further discussion; and (3) to serve as a tutorial for people to get started on guarded interaction trees.

2 Backgrounds

We start by examining the theoretical and technical backgrounds of the project. We briefly review some important features of the Iris framework, the logical relations proof method, and the theory of guarded interaction trees. Pointers to beginner-friendly guided tutorial, technical treatments in full details, and classical literatures of historical importance will also be provided.

2.1 Iris: a framework of separation logics

Iris [Jun+18] is a framework for defining higher-order concurrent separation logics mechanized in the Rocq proof assistant. In this subsection, we recall some crucial features of Iris that enables us to build a program logic for guarded interaction trees. Our presentation are informal and over-simplified. Accurate description of Iris can be found in the "Iris from the Ground Up" paper [Jun+18] and the latest Iris technical reference [Dev26]. Readers who are unfamiliar with Iris may consult the Iris lecture notes [BB23] and the software-foundations-style Iris tutorial [Gre+] to understand the fundamental concepts of concurrent separation logics and build familiarity on proof engineering with Iris proof mode inside Rocq.

2.1.1 Synthetic step-indexing

Step-indexing techniques were introduced to model general recursion and recursive types. Iris internalizes step-indexing via the later modality ($\triangleright$) to avoid explicit step arithmetic and inequality side conditions. In Iris, recursive propositions, notably the weakest preconditions, can be defined by guarded recursion, which is

[1] The Rocq development is available at `https://github.com/logsem/gitrees/pull/6`

justified by a variant of the Banach's fixedpoint theorem. To prove a recursively defined proposition, one utilizes the Löb rule, which amounts to induction on the underlying step-index.

$$\text{▷-MONO}\ \frac{P \vdash Q}{\triangleright P \vdash \triangleright Q} \qquad \text{▷-WEAK}\ \frac{P \vdash Q}{P \vdash \triangleright Q} \qquad \text{T-FIX}\ \frac{\Gamma, x : \tau \vdash t : \tau \qquad \text{variable } x \text{ can only appear in } t \text{ under } \triangleright}{\Gamma \vdash \mu x.t : \tau}$$

$$\text{EQ-FIX}\ \frac{}{\Gamma \vdash \mu x.\ t = t[(\mu x.\ t)/x] : \tau} \qquad \text{Löb}\ \frac{}{\triangleright P \Rightarrow P \vdash P}$$

The idea of having a modality for recursion appears to be invented by Nakano [Nak00], though the terminology and notations differ from that of Iris. The now-familiar synthetic step-indexing which one finds in Iris was proposed in Birkedal et al. [Bir+12]. In their paper, the *topos of trees*, i.e. the category of presheaves on the first infinite ordinal, was identified as a model of synthetic guarded domain theory (SGDT). The topos of trees gives rise to an intuitionistic higher-order logic with the later modality and a dependent type theory with a later operator. Following their work, many dependent type theories have been developed [BM13; Biz+16; Bir+19] to extend SGDT. Iris borrowed heavily from SGDT, even though the model of Iris is not based on the topos of trees or its generalizations.

2.1.2 Solving recursive domain equations

Self-referential structures, including the model of Iris itself, are defined as solutions of recursive domain equations in the realm of *ordered family of equivalences* (OFEs). The category **OFE** consists of *ordered family of equivalences* and *non-expansive* functions. Informally, an OFE $(T, (\overset{n}{=}_T)_n)$ is a set equipped with a family of step-indexed equivalence relations that become increasingly finer as the step become larger. A morphism $f : (S, (\overset{n}{=}_S)_n) \overset{\text{n.e.}}{\to} (T, (\overset{n}{=}_T)_n)$ is a set-theoretic function $f : S \to T$ that respects the equivalences. That is, if $x, y \in S$ are equivalent up to n steps, then $f(x), f(y) \in T$ must be equivalent at least up to n steps. After defining the category **OFE**, one goes on to identify some important structures (1) a subcategory **COFE** of *complete OFEs* (COFEs), (2) a special class of morphisms known as the *contractive* functions, and (3) *locally contractive bifunctors* from the category of COFEs to itself. A recursive domain equation is formulated as the solution to $G(T^{op}, T) \cong T$ where $G : \mathbf{COFE}^{op} \times \mathbf{COFE} \to \mathbf{COFE}$ is a bifunctor. Under the conditions that G is locally contractive and that $G(\mathbf{1}, \mathbf{1})$ is inhabited, one can apply the America and Rutten's theorem to construct the unique solution to the equation and the fold/unfold isomorphism [Jun+18].

2.1.3 Affine resource ownership and the frame rule

As a separation logic, Iris admits the weakening structural rule but limits the contraction rule to persistent propositions. That is, Iris propositions are generally non-duplicable but can be discarded freely. As a result, they are well-suited for modeling affine resource ownership, which is ubiquitous in programming languages. Moreover, reasoning and manipulation of resources in Iris are localized. For example, to update a specific heap cell, one do not need to consider the state of the entire heap. This informal and intuitive idea is captured by the frame rule and the *frame-preserving updates* discipline, the enabler of modular proofs. The proof rules are justified by the semantic interpretation of Iris propositions as step-indexed subsets of resources that are (1) downwards-closed with respect to the step, meaning that $(r, n) \in [\![P]\!]$ implies that $(r, m) \in [\![P]\!]$ for all index m such that $m < n$ and (2) upwards-closed with respect to resource composition, meaning that $(r, n) \in [\![P]\!]$ implies that $(r', n) \in [\![P]\!]$ if $r' = r \cdot s$ for some resource s.

$$\text{*-WEAK}\ \frac{}{P_1 * P_2 \vdash P_1} \qquad \text{*-INTRO}\ \frac{P_1 \vdash Q_1 \qquad P_2 \vdash Q_2}{P_1 * P_2 \vdash Q_1 * Q_2} \qquad \text{WP-MONO}\ \frac{\forall v.\ \Phi(v) \vdash \Psi(v)}{\mathsf{wp}\ e\ \{\Phi\} \vdash \mathsf{wp}\ e\ \{\Psi\}} \qquad \text{WP-FRAME}\ \frac{}{P * \mathsf{wp}\ e\ \{\Phi\} \vdash \mathsf{wp}\ e\ \{P * \Phi\}}$$

2.1.4 Invariants

An invariant is an Iris proposition that remains valid following its first establishment. Invariants are persistent logical resources, making it possible to be duplicated and shared across concurrent threads. Iris provides a special mechanism for accessing invariants in an atomic program step. After each atomic access, the invariant must be restored, reflecting the intuition that a shared resource may be safely accessed in an atomic operation

provided that the operation preserves the resource's validity.

$$\text{PERSISTENT-ELIM}\quad \frac{}{\Box P \vdash P} \qquad \text{PERSISTENT-DUP}\quad \frac{}{\Box P \vdash \Box P * \Box P} \qquad \text{INV-PERSISTENT}\quad \frac{}{\boxed{P}^{\iota} \vdash \Box \boxed{P}^{\iota}} \qquad \text{INV-ALLOC}\quad \frac{}{\triangleright P \vdash \Rrightarrow \boxed{P}^{\iota}}$$

$$\text{WP-ATOMIC}\quad \frac{\triangleright P \vdash \mathsf{wp}\, e\, \{v.\ \triangleright P * \Phi(v)\} \qquad \mathrm{atomic}(e)}{\boxed{P}^{\iota} \vdash \mathsf{wp}\, e\, \{\Phi\}}$$

2.1.5 Custom resource algebras

Arguably the most powerful feature of Iris is the ability for users to extend the logic by defining new resource algebras. Iris uses ghost states over resource algebras to keep track of physical and logical states of programs to facilitate reasoning. As demonstrated by Jung et al. [Jun+18], many fundamental concepts in concurrent separation logic do not have to be introduced as primitives, instead they can be defined as ghost ownership of carefully crafted resource algebras. Instances include the points-to predicate, and (surprisingly) invariants.

$$\text{VALID-ALLOC}\quad \frac{a \in \mathcal{V}_M}{\mathsf{True} \vdash \Rrightarrow \exists \gamma.\ \boxed{a}^{\gamma}} \qquad \text{OWN-VALID}\quad \frac{}{\boxed{a}^{\gamma} \vdash a \in \mathcal{V}_M} \qquad \text{OWN-OP}\quad \frac{}{\boxed{a}^{\gamma} * \boxed{b}^{\gamma} \dashv\vdash \boxed{a \cdot b}^{\gamma}} \qquad \text{OWN-UPD}\quad \frac{a \rightsquigarrow_M b}{\boxed{a}^{\gamma} \dashv \Rrightarrow \boxed{b}^{\gamma}}$$

The Iris Rocq development ships with a bunch of reusable resource algebra combinators, e.g. option RA, product RA, finite partial map RA, authoritative RA, exclusive RA, helping to avoid defining RAs completely from scratch.

2.2 The logical relations proof method

Logical relations is a prominent method to proving semantic properties for typed programming languages. The flexible method can be used to establish soundness of a type system i.e. execution of well-typed programs never gets stuck or result in an errorneous state or investigate specific programs, for example, proving a program safely encapsulate unsafe operations or proving contextual equivalence of two programs. Due to limited time for writing, we cannot include a full introduction to the proof methods in this report. Readers who do not have sufficient knowledge of logical relations can read Skorstengaard's excellent tutorial [Sko19]. The semantics course note by Dreyer et al. [Dre+25] is another excellent learning resource. It covers all of the techniques we will discuss in this subsection.

2.2.1 A brief history of logical relations

The logical relations method has been historically known as *Tait's computability method* and *Girard's reducibility candidates*, as it was pioneered by Tait and Girard. Tait [Tai67] first employed the method to prove a normalization property of Gödel's System T, which can be seen as the simply typed λ-calculus extended with natural numbers and primitive recursion. Girard [Gir72] extended the method to prove strong normalization for the second-order polymorphic λ-calculus, also known as System F. The primary novelty in Girard's proof was the interpretation of impredicative polymorphism by quantifying over *candidats de réductibilité* instead of all syntactic types to prevent circularity. Subsequently, the power of logical relations was greatly expanded due to Reynolds's contribution. Reynolds [Rey83] developed the notion of *relational parametricity* for data abstractions and showed that polymorphic functions in System F are parametric, i.e., they treat inputs of different types uniformly, hence the name parametricity.

While the proof method was immensely successful for pure and total languages, it was not easy to account for recursion and dynamic allocation, two ubiquitous yet complicated features in realistic programming languages. To treat the two aforementioned features, Kripke logical relations and step-indexing techniques were introduced. Coquand and Gallier [CG90], inspired by the Kripke semantics of intuitionistic logic, interpreted typing contexts as frames and context extension as accessible worlds. Their interpretation yielded a simple proof of the strong normalization of the Calculus of Constructions (CoC). Later, O'Hearn and Pitts [OT93; Pit96] adapted Kripke logical relations to model programming languages with local variables. The other addition, step-indexing, was originally introduced by Appel and McAllester [AM01] to build an operational semantics-based models for STLC extended with recursive types. They include step counters to introduces stratification in recursive structures. Their logical relations can be used to soundly prove safety and observational equivalence of programs without resorting to a full-scale traditional domain-theoretic treatment, which is advantageous in the context of foundational proof carrying code (PCC) systems. The techniques were eventually combined, giving rise to step-indexed Kripke logical relations (SKLR). Ahmed's PhD thesis [Ahm04] showed how to use SKLR to model feature-rich realistic programming languages.

Despite being a scalable and powerful method, SKLR has some drawbacks. Proofs are often obscured and hard to mechanize due to the involvement of global state manipulations on the Kripke frame and tedious step-index arithmetics. These complications can be drastically reduced by defining logical relations in a suitable metatheory that can handle step-indexing and frame-preserving state manipulation abstractly. Iris is such a metatheory. Krebbers et al. [KTB17; Kre+18] demonstrated how to define logical relations in Iris and carry out mostly straight-forward proofs in Iris Proof Mode. Compared to raw SKLRs, Iris-based logical relations proofs are often much more intuitive to understand on paper and easier to formalize in Rocq.

2.2.2 Logical relations in Iris

To illustrate logical relations in Iris, we give a proof sketch for the type soundness of System F with reference types and recursive types. We highlight how Iris features are used to simplify SKLRs. What we will demonstrate in this section largely follows from the case study in [BB23; Tim+24].

Logical relations for closed terms. A typing judgement has the form $\Xi; \Gamma \vdash e : \tau$, where Ξ is the free type variables, Γ is the free term variables, and τ is a valid type under Ξ. The term e is said to be closed when $\Gamma = \bullet$ is empty, i.e., it involves no free term variable. We define the value relation $[\![\Xi \vdash \tau]\!]_\Delta : \mathrm{Val} \to \mathit{iProp}_\square$ and the expression relation $[\![\Xi \vdash \tau]\!]^{\mathcal{E}}_\Delta : \mathrm{Expr} \to \mathit{iProp}_\square$ by induction on the type τ. Here $\delta : \Xi \to (\mathrm{Val} \to \mathit{iProp}_\square)$ is the interpretation of free type variables. The expression relation is defined as $[\![\Xi \vdash \tau]\!]^{\mathcal{E}}_\Delta(e) \triangleq \mathsf{wp}\, e\, \{[\![\Xi \vdash \tau]\!]_\Delta\}$, stating that e will not get stuck and may terminate as a value in $[\![\Xi \vdash \tau]\!]_\Delta$. The value relation, capturing concerns the shape and hereditary semantic property of a value of the type in question, is defined as follows:

$$
\begin{aligned}
[\![\Xi \vdash \alpha]\!]_\Delta(v) &\triangleq \Delta(\alpha)(v) \\
[\![\Xi \vdash 1]\!]_\Delta(v) &\triangleq v = () \\
[\![\Xi \vdash \mathbb{N}]\!]_\Delta(v) &\triangleq \exists n \in \mathbb{N}.\ v = n \\
[\![\Xi \vdash \mathbb{B}]\!]_\Delta(v) &\triangleq v \in \{\mathsf{true}, \mathsf{false}\} \\
[\![\Xi \vdash \tau_1 \times \tau_2]\!]_\Delta(v) &\triangleq \exists v_1, v_2.\ v = (v_1, v_2) * [\![\Xi \vdash \tau_1]\!]_\Delta(v_1) * [\![\Xi \vdash \tau_2]\!]_\Delta(v_2) \\
[\![\Xi \vdash \tau_1 + \tau_2]\!]_\Delta(v) &\triangleq \bigvee_{i \in \{1,2\}} (\exists w.\ v = \mathsf{inj}_i\, w * [\![\Xi \vdash \tau_i]\!]_\Delta(w)) \\
[\![\Xi \vdash \tau \to \tau']\!]_\Delta(v) &\triangleq \square\left(\forall w.\ [\![\Xi \vdash \tau]\!]_\Delta(w) \mathrel{-\!\!*} [\![\Xi \vdash \tau']\!]^{\mathcal{E}}_\Delta(v\ w)\right) \\
[\![\Xi \vdash \forall \alpha.\ \tau]\!]_\Delta(v) &\triangleq \square\left(\forall (\Psi : \mathrm{Val} \to \mathit{iProp}_\square).\ [\![\Xi, \alpha \vdash \tau]\!]^{\mathcal{E}}_{\Delta, \alpha \mapsto \Psi}(v\ \langle\rangle)\right) \\
[\![\Xi \vdash \exists \alpha.\ \tau]\!]_\Delta(v) &\triangleq \square\left(\exists (\Psi : \mathrm{Val} \to \mathit{iProp}_\square).\ \exists w.\ (v = \mathsf{pack}\, \langle w \rangle) * [\![\Xi, \alpha \vdash \tau]\!]^{\mathcal{E}}_{\Delta, \alpha \mapsto \Psi}(w)\right) \\
[\![\Xi \vdash \mu \alpha.\ \tau]\!]_\Delta(v) &\triangleq \mu(\Psi : \mathrm{Val} \to \mathit{iProp}_\square).\ \exists w.\ (v = \mathsf{fold}\, w) * \triangleright [\![\Xi \vdash \tau]\!]_{\Delta, \alpha \mapsto \Psi}(w) \\
[\![\Xi \vdash \mathsf{ref}(\tau)]\!]_\Delta(v) &\triangleq \exists \ell.\ v = \ell * \boxed{\exists w.\ \ell \mapsto w * [\![\Xi \vdash \tau]\!]_\Delta(w)}^{\mathcal{N}.\ell}
\end{aligned}
$$

We here remind the reader some technical details for ensuring the well-foundedness of the relations:

1. By working in a separation logic with local resource manipulation, we no longer need to include explicit Kripke frames.

2. The expression relation is closed under head expansion. That is, $e \mapsto e'$ and $[\![\Xi \vdash \tau]\!]^{\mathcal{E}}_\Delta(e')$ implies $[\![\Xi \vdash \tau]\!]^{\mathcal{E}}_\Delta(e)$. This is one of the requirement for a relation to be a reducibility candidate.

3. Quantifying over reducibility candidates. When interpreting $\forall \alpha.\ \tau$ and $\exists \alpha.\ \tau$, we did not directly quantify over all syntactic types, which would introduce circularity. Instead, we quantify over the type of value relations, i.e., $\mathrm{Val} \to \mathit{iProp}_\square$. We further note that impredicativity of Iris is crucial for the two clauses to be acceptable.

4. Recursion (at the object-level) as guarded recursion (at the meta-level). We interpret a recursive type as a guarded fixed point, offloading step-indexing to Iris.

5. Environment-based Interpretation. It is necessary that we interpret types relative to the interpretation of free type variables. Carrying out syntactical substitution as soon as a type variable is instantiated is not a valid option. This is because that the substitution result $\tau[\beta/\alpha]$ is generally not structurally smaller than α, meaning that one cannot use $[\![\Xi \vdash \tau[\beta/\alpha]]\!]$ to define $[\![\Xi \vdash \forall \alpha.\tau]\!]$.

6. Operational characterization of well-typed programs is absorbed into Hoare triples and/or weakest pre-conditions propositions.

We further note that the relations are persistent. That is, they can be copied or discarded with no restriction at all. This is what we should expect since type system of the programming language admits unlimited weakening and contraction.

Lifting to open terms. We define the semantically well-typed relation by lifting the expression relation to open terms. Informally, the proposition $\Xi \mid \Gamma \vDash e : \tau$ states that "whenever a semantically well-formed closing substitution γ instantiating Γ is provided, the substitution result $\gamma(e)$ is in the expression relation $[\![\Xi \vdash \bullet]\!]^{\mathcal{E}}_{\tau}$". To define this relation, we additionally introduce the closing substitution relation $[\![\Xi \vdash \Gamma]\!]^{\mathcal{G}}_{\Delta}\gamma$ to capture the property that γ instantiate all free variable in Γ with a closed values in the corresponding value relation.

$$[\![\Xi \vdash \Gamma]\!]^{\mathcal{G}}_{\Delta}(\gamma) \triangleq \mathop{\Large *}_{(x \mapsto v) \in \gamma} [\![\Xi \vdash \Gamma(x)]\!]_{\Delta}(v)$$

$$\Xi \mid \Gamma \vDash e : \tau \triangleq \forall \gamma.\ [\![\Xi \vdash \Gamma]\!]^{\mathcal{G}}_{\Delta}(\gamma) \mathrel{-\!\!*} [\![\Xi \vdash \tau]\!]^{\mathcal{E}}_{\Delta}(\gamma(e))$$

Fundamental Lemma. We prove that the semantic typing relation is compatible with the syntactic typing rules. For example, the typing rules for λ-abstraction and function application correspond to the following two lemmas

$$\textsc{T-Abs}\ \frac{\Xi \mid \Gamma, x : \tau_1 \vdash e : \tau_2}{\Xi \mid \Gamma \vdash \lambda x : \tau_1.\ e : \tau_1 \to \tau_2} \qquad \textsc{T-App}\ \frac{\Xi \mid \Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Xi \mid \Gamma \vdash e_2 : \tau_1}{\Xi \mid \Gamma \vdash e_1\ e_2 : \tau_2}$$

$$\textsc{compat-Abs}\ \frac{\Xi \mid \Gamma, x : \tau_1 \vDash e : \tau_2}{\Xi \mid \Gamma \vDash \lambda x : \tau_1.\ e : \tau_1 \to \tau_2} \qquad \textsc{compat-App}\ \frac{\Xi \mid \Gamma \vDash e_1 : \tau_1 \to \tau_2 \qquad \Xi \mid \Gamma \vDash e_2 : \tau_1}{\Xi \mid \Gamma \vDash e_1\, e_2 : \tau_2}$$

By induction on the typing derivation, we can show that all syntactically well-typed term is semantically well-typed. This result is often being referred to as the fundamental lemma in the literature.

$$\forall \Xi, \Gamma, e, \tau.\ \Xi \mid \Gamma \vdash e : \tau \mathrel{-\!\!*} \Xi \mid \Gamma \vDash e : \tau$$

Specialize the result with $\Xi = \bullet$ and $\Gamma = \bullet$, we get to conclude that all well-typed closed programs are in the expression relation, which is a weakest precondition.

Adequacy. The final step is to show that the logical relation implies the safety property in question. We have already derived that each well-typed programs have a weakest precondition governing their safety. The type soundness proof can be finished by invoking the adequacy of weakest preconditions and the Iris adequacy theorem.

2.3 Denotational semantics

Denotational semantics is an approach of defining a formal semantics of a programming language. While an operational semantics specifies the rules for executing programs, a denotational semantics characterizes what a program computes by defining a compositional map from the syntax to a domain of semantic objects. By interpreting a language in a domain that is inherently extensional, we can easily prove many equivalences that are challenging to prove in operational models. One of such classical example is congruence of contextual equivalence, i.e., to derive $C[e_1] \cong C[e_2]$ given that $e_1 \cong e_2$ are equivalent terms.

The relationship between a denotational model $[\![\cdot]\!]$ and the operational semantics is traditionally characterized by the following properties:

Soundness $e_1 \mapsto^* e_2 \implies [\![e_1]\!] = [\![e_2]\!]$ The denotational semantics is stable under reductions defined by the operational semantics.

Computational Adequacy For base type expression e and value v, $[\![e]\!] = [\![v]\!] \implies e \Downarrow v$. If the denotation of a term equals to that of a base type value, then the term computes to the value operationally.

Full Abstraction $e_1 \cong_{ctx} e_2 \iff [\![e_1]\!] = [\![e_2]\!]$ Contextual equivalence, which is defined by the operational semantics, and denotational equivalence, which is defined by equality in the semantic domain, coincide.

In this project, soundness and computational adequacy are our primary focus. The Stanford Encyclopedia of Philosophy has a page on game semantics [Car21] which contains a detailed discussion of the connection between denotational semantics and operational semantics We refer readers who are unfamiliar with (domain-theoretic) denotational semantics to the lecture notes by Pitts et al. [Len+24] and the chapter on domain theory by Jung et al. [Jun+96] in the *handbook of logic in computer science* for a comprehensive treatment of classical domain theory.

2.4 Guarded Interaction Trees

To develop an adequate denotational semantics for a programming language, a semantic domain has to be decided first. Guarded interaction trees [FTB24] is essentially a domain of higher-order untyped effectful programs. It is well-suited for interpreting various languages with higher-order effects. Indeed, as demonstrated in by Frumin et al. [FTB24] guarded interaction trees can be used to investigate the semantics of interoperability.

Guarded Interaction trees belongs to a long-standing tradition of modeling and reasoning about effectful programs in type theories via monads, an approach pioneered by Moggi [Mog91]. Type theories generally rely on normalization and canonicity to ensure consistency, meaning that programs and proofs in a type theory have to be total. Consequently a large class of practical language features including partial functions, non-termination, input/output effects cannot be directly programmed. Many solutions aiming at overcoming this limitation have been proposed. Capretta [Cap05] introduced the coinductive delay monad for potentially non-terminating computations. McBride's general monad [McB15] for modeling recursive function calls as request-response pairs. Piróg [PG14] generalized the delay monad to coinductive resumption monad. Recently, Xia et al. [Xia+19] devised the interaction trees (ITrees), a semantic domain for first-order effectful programs. A major limitation of these works is the lack of primitive support for higher-order functions and higher-order effects. Leveraging the synthetic step-indexing technique, GItree is able to encode higher-order features natively.

Starting from now, we implicitly work within the logic of Iris. When we say a type, we mean an OFE or a COFE. A function type should be understood as the space of non-expansive maps between two OFEs, which is itself an OFE. We now present a highly simplified overview of guarded interaction trees. While this introduction does not entirely match the GItree theory mechanized in Rocq, it provides enough foundational understanding to make the subsequent case studies accessible.

2.4.1 Effect Signature

The effects that a program can make use of is described by an *effect signature* $(E, \mathsf{Ins}, \mathsf{Outs})$. E is a finite set of available effects. For each effect $i \in E$, the two functors $\mathsf{Ins}_i, \mathsf{Outs}_i : \mathbf{COFE} \to \mathbf{COFE}$ specify the expected input and output type of effect i, respectively.

For example, we can use the following effect signature to model I/O.

$$E = \{\text{input}, \text{output}\} \quad \mathsf{Ins}_i(X) = \begin{cases} \mathbf{1} & i = \text{input} \\ \mathbb{N} & i = \text{output} \end{cases} \quad \mathsf{Outs}_i(X) = \begin{cases} \mathbb{N} & i = \mathit{input} \\ \mathbf{1} & i = \mathit{output} \end{cases}$$

The informal understanding of this signature is:

1. A program can make use of the input effect and the output effect.
2. To invoke the input effect, a unit value has to be passed as argument. When returning from the effect handler, a natural number value will be returned to the program.
3. To invoke the output effect, a natural number value has to be passed as argument. When returning from the effect handler, a unit value will be returned to the program.

The above signature is *first-order* since $Ins_i(X)$ and $Outs_i(X)$ completely ignore X. X will be eventually instantiated by the domain of (higher-order) effectful programs, enabling us to express higher-order effects, i.e., effects that can take programs as inputs and output programs. To illustrate higher-order effects, consider the following effect signature for (higher-order) store.

$$E = \{\text{alloc}, \text{dealloc}, \text{read}, \text{write}\} \quad \mathsf{Ins}_i(X) = \begin{cases} X & i = \text{alloc} \\ \text{Loc} & i = \text{dealloc} \\ \text{Loc} & i = \text{read} \\ \text{Loc} \times X & i = \text{write} \end{cases} \quad \mathsf{Outs}_i(X) = \begin{cases} \text{Loc} & i = \text{alloc} \\ \mathbf{1} & i = \text{dealloc} \\ X & i = \text{read} \\ \mathbf{1} & i = \text{write} \end{cases}$$

We note that the effect read may return a runnable program and the effect write asks for a value in the domain of higher-order effectful program.

2.4.2 Guarded interaction trees as a fixed point

Fixing a type Error of errors, a signature E of available effects, and a type A of base values, we seek a shallow embedding of the domain D of higher-order untyped effectful programs satisfying the following recursive domain equation:

$$D \cong \mathrm{Error} + A + \blacktriangleright D + \blacktriangleright[D \to D] + \sum_{i \in E} (\mathsf{Ins}_i(\blacktriangleright D) \times [\mathsf{Outs}_i(\blacktriangleright D) \to \blacktriangleright D])$$

That is, a program in D can be:

1. An error,
2. A base value,
3. A function that takes a program and returns a program, or
4. An invocation of an (higher-order) effect along with the data to be transferred to the effect handler and a continuation to be resumed upon returning from the effect handler.

Notice that we reuse the function type of the metatheory to define higher-order function of the object language. As a result, function abstraction and application are not syntactic constructs in the object language. Further, function application is evaluated at the meta level.

D cannot be defined inductively nor coinductively. We rephrase the mixed variance recursive domain equation as the fixed point of a bifunctor $F : \mathbf{COFE}^{op} \times \mathbf{COFE} \to \mathbf{COFE}$, where

$$F(X, Y) = \mathrm{Error} + A + \blacktriangleright Y + \blacktriangleright[X \to Y] + \sum_{i \in E} (\mathsf{Ins}_i(\blacktriangleright Y) \times [\mathsf{Outs}_i(\blacktriangleright X) \to \blacktriangleright Y])$$

It turns out that F is locally contractive and that $F(\mathbf{1}, \mathbf{1})$ is non-empty. Therefore, the equation $F(D, D) \cong D$ has a unique solution. We call the solution type guarded interaction trees, denoted by $\mathrm{IT}_E(A)$. The recursive domain equation solver in Iris allows us to derive the fold/unfold isomorphism for $\mathrm{IT}_E(A)$ and a recursive elimination principle, upon which the following definitions and equations can be derived:

- When the base type and effect signature are of no significance or can be inferred from the context, we write IT for $\mathrm{IT}_E(A)$.
- For $\alpha \in \mathrm{IT}$, we write $\mathrm{Tick}(\alpha) \triangleq \mathrm{Tau}(\mathrm{next}\,\alpha)$. We note that $\mathrm{Tick}(\alpha)$ is the head normal form of Tau GItree.
- A collection of smart constructors for GItrees can be defined using the fold morphism.

$$\begin{aligned}
&\mathrm{Ret} : A \to \mathrm{IT}\\
&\mathrm{Err} : \mathrm{Error} \to \mathrm{IT}\\
&\mathrm{Tau} : \blacktriangleright\mathrm{IT} \to \mathrm{IT}\\
&\mathrm{Fun} : \blacktriangleright(\mathrm{IT} \to \mathrm{IT}) \to \mathrm{IT}\\
&\mathrm{Vis} : \forall(\mathsf{op} : I)(\mathsf{inputs} : \mathsf{Ins}_i(\blacktriangleright\mathrm{IT}))(\mathsf{callback} : \mathsf{Outs}_i(\blacktriangleright\mathrm{IT}) \to \blacktriangleright\mathrm{IT}) \to \mathrm{IT}
\end{aligned}$$

- We assume that the base value type A is a coproduct of the form $A_1 + A_2 + A_3 \cdots + A_n$. Given an injection $\mathsf{inj} : B \hookrightarrow A$, we write $\mathrm{Ret}(b)$ for $\mathrm{Ret}(\mathsf{inj}\, b)$ for every $b : B$.
- We call a GItrees of the form $\mathrm{Ret}(a)$ or $\mathrm{Fun}(f)$ a value. We use Greek letters with v-subscript, e.g., α_v, for GItree value meta variables.
- A function $\mathrm{IT}_E(A) \to \mathrm{IT}_E(A)$ is called a homomorphism if it satisfies the following equations:

$$f(\mathrm{Err}(e)) \equiv \mathrm{Err}(e) \quad f(\mathrm{Tick}(\alpha)) \equiv \mathrm{Tick}(f(\alpha)) \quad f(\mathrm{Vis}_i(x, k)) \equiv \mathrm{Vis}_i(x, \blacktriangleright f \circ k)$$

  The identity function is a homomorphism and the composition of homomorphisms is a homomorphism.
- We can define three functions get_ret, get_fun, and get_val satisfying the following equations.

$$\begin{aligned}
\mathrm{get_ret}(f)(\mathrm{Ret}(n)) &\equiv f\ n & \mathrm{get_ret}(f)(\mathrm{Err}(e)) &\equiv \mathrm{Err}(e)\\
\mathrm{get_ret}(f)(\mathrm{Tick}(\alpha)) &\equiv \mathrm{Tick}(\mathrm{get_ret}(f)(\alpha)) & \mathrm{get_ret}(f)(\mathrm{Vis}_i(x, k)) &\equiv \mathrm{Vis}_i(x, \blacktriangleright \circ \mathrm{get_ret}(f) \circ k)\\
\mathrm{get_fun}(g)(\mathrm{Fun}(h)) &\equiv g\ h & \mathrm{get_fun}(g)(\mathrm{Err}(e)) &\equiv \mathrm{Err}(e)\\
\mathrm{get_fun}(g)(\mathrm{Tick}(\alpha)) &\equiv \mathrm{Tick}(\mathrm{get_fun}(g)(\alpha)) & \mathrm{get_fun}(g)(\mathrm{Vis}_i(x, k)) &\equiv \mathrm{Vis}_i(x, \blacktriangleright \circ \mathrm{get_fun}(g) \circ k)\\
\mathrm{get_val}(h)(\mathrm{Ret}(n)) &\equiv h(\mathrm{Ret}(n)) & \mathrm{get_val}(h)(\mathrm{Fun}(f)) &\equiv h(\mathrm{Fun}(f))\\
\mathrm{get_val}(h)(\mathrm{Tick}(\alpha)) &\equiv \mathrm{Tick}(\mathrm{get_val}(h)(\alpha)) & \mathrm{get_val}(h)(\mathrm{Vis}_i(x, k)) &\equiv \mathrm{Vis}_i(x, \blacktriangleright \circ \mathrm{get_val}(h) \circ k)
\end{aligned}$$

- We can define the "call-by-name" function application combinator as well as the "call-by-value" version. The combinators satisfy the following equations.

$$\begin{aligned} \mathrm{Fun}(\mathrm{next}\, f) \bullet_{\mathrm{cbn}} \beta &\equiv f(\beta) & \mathrm{Tick}(\alpha) \bullet_{\mathrm{cbn}} \beta &\equiv \mathrm{Tick}(\alpha \bullet_{\mathrm{cbn}} \beta) \\ \alpha_v \bullet_{\mathrm{cbv}} \beta &\equiv \alpha_v \bullet_{\mathrm{cbn}} \beta & \mathrm{Tick}(\alpha) \bullet_{\mathrm{cbv}} \beta_v &\equiv \mathrm{Tick}(\alpha \bullet_{\mathrm{cbv}} \beta_v) \\ \mathrm{Err}(e) \bullet_{\mathrm{cbn}} \beta &\equiv \mathrm{Err}(e) & \mathrm{Err}(e) \bullet_{\mathrm{cbv}} \beta_v &\equiv \mathrm{Err}(e) \end{aligned}$$

  We note that the CBV function application combinator $- \bullet -$ adopts a "right-to-left" evaluation order.

- Conditional combinator $\mathrm{If} : \mathrm{IT} \to \mathrm{IT} \to \mathrm{IT} \to \mathrm{IT}$ satisfying:

$$\begin{aligned} \mathrm{If}(\mathrm{Ret}(0))(\alpha)(\beta) &\equiv \beta \\ \mathrm{If}(\mathrm{Ret}(n+1))(\alpha)(\beta) &\equiv \alpha \\ \mathrm{If}(\mathrm{Tick}(\gamma))(\alpha)(\beta) &\equiv \mathrm{Tick}(\mathrm{If}(\gamma)(\alpha)(\beta)) \\ \mathrm{If}(\mathrm{Vis}_i(x,k))(\alpha)(\beta) &\equiv \mathrm{Vis}_i(x, \blacktriangleright \circ \mathrm{If}(-)(\alpha)(\beta) \circ k) \end{aligned}$$

- Natural number operation combinator $\mathrm{NatOp}_\oplus : \mathrm{IT} \to \mathrm{IT} \to \mathrm{IT}$ for $\oplus \in \{+, -, *\}$:

$$\begin{aligned} \mathrm{NatOp}_\oplus(\mathrm{Ret}(n))(\mathrm{Ret}(m)) &\equiv \mathrm{Ret}(n \oplus m) \\ \mathrm{NatOp}_\oplus(\mathrm{Tick}(\gamma))(\beta) &\equiv \mathrm{Tick}(\mathrm{NatOp}_\oplus(\gamma)(\beta)) \\ \mathrm{NatOp}_\oplus(\mathrm{Vis}_i(x,k))(\beta) &\equiv \mathrm{Vis}_i(x, \blacktriangleright \circ \mathrm{NatOp}_\oplus(-)(\beta) \circ k) \end{aligned}$$

- $\mathrm{get\_fun}(-, f)$, $\bullet_{\mathrm{cbn}}(-, \beta)$, $\alpha \bullet_{\mathrm{cbv}} -$, $- \bullet_{\mathrm{cbv}} \beta_v$, $\mathrm{NatOp}_\oplus(-)(\beta)$ and $\mathrm{NatOp}_\oplus(\alpha_v)(-)$ are GItree homomorphisms.

- While-loop combinator $\mathrm{While} : \mathrm{IT} \to \mathrm{IT} \to \mathrm{IT}$ defined via guarded recursion:

$$\mathrm{While}(\alpha)(\beta) \triangleq \mathrm{If}(\alpha)(\beta \bullet_{\mathrm{cbv}} \mathrm{Tick}(\mathrm{While}(\alpha)(\beta)))(\mathrm{Ret}(()))$$

- The let-in combinator $\mathrm{LET}\ x = \alpha\ \mathrm{in}\ \beta \triangleq \mathrm{get\_val}(\alpha, \lambda x.\ \beta(x))$.

- The sequencing combinator $e_1;\ e_2 \triangleq \mathrm{get\_val}(\alpha, \lambda\_.\ \beta)$.

Again, the combinators are metalevel definitions instead of being part of the language of guarded interaction trees.

2.4.3 Effect reifiers, GItree reductions, and program logics

Having defined the language of GItrees and some meta-level GItree combinators, we will define an operational semantics of GItrees that actually evaluate effect invocation by using *effect reifiers*. A reifier specifies a semantics of an effect signature $(E, \mathsf{Ins}, \mathsf{Outs})$ by defining the internal state of the effect $\mathit{State}(X)$ and the handler for each effect:

$$r : \prod_{i \in E} \left(\mathsf{Ins}_i(X) \times \mathit{State}(X) \to (\mathsf{Outs}_i(X) \times \mathit{State}(X))^? \right)$$

The parameter X will be eventually instantiated with $\blacktriangleright \mathrm{IT}_E(A)$.

One possible interpretation of the input/output effect is

- The state is a pair of list, representing the input and the output tape, respectively. $\mathit{State}(X) = \mathbb{N}^* \times \mathbb{N}^*$.
- The reifier for input: $r_{\mathrm{input}}(l_0, l_1) = \begin{cases} \mathrm{None} & l_0 = [] \\ \mathrm{Some}(x, (l_0', l_1)) & l_0 = x : l_0' \end{cases}$
- The reifier for output is $r_{\mathrm{output}}(x, (l_0, l_1)) = \mathrm{Some}((), (l_0, x : l_1))$

Given a reifier r for E, we can define a small-step operational semantics of GItrees $\mathrm{IT}_E(A)$.

Definition 2.1 (GItree reduction step)**.** We call a pair of a GItree program and a reifier state a *GItree program configuration* $\mathrm{IT}_E(A) \times \mathit{State}(\blacktriangleright \mathrm{IT}_E(A))$. The GItree reduction step is a binary relation over configurations generated by the following two rules:

Tick step $(\mathrm{Tick}(\alpha), \sigma) \rightsquigarrow (\alpha, \sigma)$.

Reify step $(\mathrm{Vis}_i(x,k),\sigma) \rightsquigarrow (\beta,\sigma)$ when $r_i(x) = \mathrm{Some}(y,\sigma')$ and $k\ y = \mathrm{next}\,\beta$.

The operational semantics of GItrees is defined as the reflexive and transitive closure of the reduction relation.

A program logic for reasoning about the execution of guarded interaction trees can be established. The program logic for GItrees is centered around the weakest precondition $\mathsf{wp}\,\alpha\,\{\Phi\}$.

$$\frac{\alpha : \mathrm{IT}_E(A) \qquad \Phi : \mathrm{IT}^v_E(A) \to \mathit{iProp}}{\mathsf{wp}\,\alpha\,\{\Phi\} : \mathit{iProp}}$$

As usual, it is defined via a guarded fixed point, asserting that either α is already a GItree value satisfying the post-condition Φ or it steps to another GItree β for which $\mathsf{wp}\,\beta\,\{\Phi\}$ holds under suitable ghost state updates.

WP-VAL
$$\frac{\Phi(\alpha_v)}{\mathsf{wp}\,\alpha_v\,\{\Phi\}}$$

WP-TICK
$$\frac{\triangleright\,\mathsf{wp}\,\alpha\,\{\Phi\}}{\mathsf{wp}\,\mathrm{Tick}(\alpha)\,\{\Phi\}}$$

WP-BIND
$$\frac{\mathrm{IT\text{-}Hom}(f) \qquad \mathsf{wp}\,\alpha\,\{\alpha_v.\mathsf{wp}\,f(\alpha_v)\,\{\Phi\}\}}{\mathsf{wp}\,f(\alpha)\,\{\Phi\}}$$

WP-UPD
$$\frac{\Rrightarrow \mathsf{wp}\,\alpha\,\{\alpha_v.\Rrightarrow\Phi(\alpha_v)\}}{\mathsf{wp}\,\alpha\,\{\Phi\}}$$

WP-MONO
$$\frac{\mathsf{wp}\,\alpha\,\{\Psi\} \qquad \forall\alpha_v.\Psi(\alpha_v) \mathrel{-\!\!*} \Phi(\alpha_v)}{\mathsf{wp}\,\alpha\,\{\Phi\}}$$

WP-REIFY
$$\frac{\mathsf{has\_state}(\sigma) \qquad r_i(x,\sigma) = \mathrm{Some}(\beta,\sigma') \qquad \triangleright\,(\mathsf{has\_state}(\sigma') \mathrel{-\!\!*} \mathsf{wp}\,\beta\,\{\Phi\})}{\mathsf{wp}\,\mathrm{Vis}_i(x,k)\,\{\Phi\}}$$

These proof rules should be mostly familiar. GItree homomorphisms can be viewed as evaluation contexts. The WP-VAL rule is the monadic return and the WP-BIND rule is the monadic bind. We note that the context-independent reification rule takes a premise $\mathsf{has\_state}(\sigma)$, which asserts exclusive and complete ownership of the reifier state. However, it is often desirable to reason about effects with partial knowledge of the state. Currently, we cannot derive the following WP-READ and WP-WRITE rules for the higher-order store effect:

WP-READ'
$$\frac{l \hookrightarrow \alpha \qquad \triangleright(l \hookrightarrow \alpha \mathrel{-\!\!*} \mathsf{wp}\,\alpha\,\{\Phi\})}{\mathsf{wp}\,(!l)\,\{\Phi\}}$$

WP-STORE'
$$\frac{l \hookrightarrow \alpha \qquad \triangleright(l \hookrightarrow \beta \mathrel{-\!\!*} \Phi(\mathrm{Ret}()))}{\mathsf{wp}\,(l \leftarrow \beta)\,\{\Phi\}}$$

To recover this kind of reasoning principles, one has to design an accompanying ghost theory, which will be exemplified later sections.

Building on the adequacy theorem of Iris, a soundness theorem of the GItree program logic with respect to GItree reduction semantics can be proved:

Theorem 2.1 (GItree program logic soundness)**.** *If* $\mathsf{has\_state}\sigma \vdash \mathsf{wp}\,\alpha\,\{\Phi\}$ *is derivable, where* Φ *is a pure Iris proposition, then for every* β *and* σ' *such that* $(\alpha,\sigma) \rightsquigarrow^* (\beta,\sigma')$ *the following disjunctive statement holds.*

- *either* β *is a GItree value and* $\Phi(\beta)$ *hold in the metalogic where Iris is defined;*
- *or* $(\beta,\sigma') \rightsquigarrow (\beta_1,\sigma_1)$ *for some* β_1 *and* σ_1*.*

Informally, it states that $\mathsf{wp}\,\alpha\,\{\Phi\}$ ensures that α is safe to run in the sense that it never gets stuck before reducing to a value. As a corollary α never reduces to an erroneous GItree of the shape $\mathrm{Err}(\cdot)$, since the later is not a value and cannot be further reduced.

2.4.4 Combining effects

The domain of guarded interaction trees $\mathrm{IT}_E(A)$ is parameterized on a single effect signature. To enable constructing and reasoning about programs that uses multiple effects, a mechanism for composing effect is required.

Given two effects $(E_1, \mathsf{Ins}^1, \mathsf{Outs}^1)$ and $(E_2, \mathsf{Ins}^2, \mathsf{Outs}^2)$, one can combine them orthogonally. The combined signature is defined as $(E, \mathsf{Ins}, \mathsf{Outs})$, where

$$E \triangleq E_1 + E_2 = \{(1,i) \mid i \in E_1\} \cup \{(2,j) \mid j \in E_2\}$$

$$\mathsf{Ins}_e(X) \triangleq \begin{cases} \mathsf{Ins}^1_i(X) & e = (1,i) \\ \mathsf{Ins}^2_j(X) & e = (2,j) \end{cases}$$

$$\mathsf{Outs}_e(X) \triangleq \begin{cases} \mathsf{Outs}^1_i(X) & e = (1,i) \\ \mathsf{Outs}^2_j(X) & e = (2,j) \end{cases}$$

A reifier for E can also be derived by combining a reifier r^1 for E_1 and a reifier r^2 for E_2. The state functor of the combined effect is given by $\mathit{State}(X) \triangleq \mathit{State}^1(X) \times \mathit{State}^2(X)$

$$r_e(t,(\sigma_1,\sigma_2)) \triangleq \begin{cases} \mathrm{Some}(t',(\sigma_1',\sigma_2)) & e=(1,i) \wedge \mathrm{Some}(t,\sigma_1') = r_i^1(t,\sigma_1) \\ \mathrm{Some}(t',(\sigma_1,\sigma_2')) & e=(2,j) \wedge \mathrm{Some}(t,\sigma_2') = r_j^2(t,\sigma_2) \\ \mathrm{None} & \text{otherwise} \end{cases}$$

Based on this reifier, we can recover the WP-REIFY program logic proof rule.

$$\textsc{wp-reify-sub-1}\ \frac{\mathsf{has\_state}^1(\sigma_1) \qquad r_i^1(x,\sigma_1) = \mathrm{Some}(\beta,\sigma_1') \qquad \triangleright\left(\mathsf{has\_state}^1(\sigma_1') \mathrel{-\!\!*} \mathsf{wp}\ \beta\ \{\Phi\}\right)}{\mathsf{wp}\ \mathrm{Vis}_{(1,i)}(x,k)\ \{\Phi\}}$$

$$\textsc{wp-reify-sub-2}\ \frac{\mathsf{has\_state}^2(\sigma_2) \qquad r_j^2(x,\sigma_2) = \mathrm{Some}(\beta,\sigma_2') \qquad \triangleright\left(\mathsf{has\_state}^2(\sigma_2') \mathrel{-\!\!*} \mathsf{wp}\ \beta\ \{\Phi\}\right)}{\mathsf{wp}\ \mathrm{Vis}_{(2,j)}(x,k)\ \{\Phi\}}$$

We can easily generalize the orthogonal combination construction to mix arbitrary number of effects. This leads to the notion of *sub-effect*. An effect signature $(E, \mathsf{Ins}, \mathsf{Outs})$ is said to be a sub-effect of another signature $(F, \mathsf{Ins}', \mathsf{Outs}')$, denoted by $E \rightarrowtail F$, if there is an inclusion $\epsilon : E \hookrightarrow F$, such that $\mathsf{Ins}_i(X) \cong \mathsf{Ins}'_{\epsilon(i)}$ and $\mathsf{Outs}_i(X) \cong \mathsf{Outs}'_{\epsilon(i)}$.

Now that a mechanism for combining effects, we consider how to combine GItrees of different effect signatures. One may expect to derive a GItree embedding $\mathrm{IT}_E(A) \to \mathrm{IT}_F(A)$ from $E \rightarrowtail F$. However this quickly proves to be impossible. The recursive domain equation governing IT is mixed-variant. Specifically, for the function case, we have $\blacktriangleright(\mathrm{IT} \to \mathrm{IT}) \hookrightarrow \mathrm{IT}$. It is unclear how to define a function of type $[\mathrm{IT}_F(A) \to \mathrm{IT}_F(A)]$ given a function of type $[\mathrm{IT}_E(A) \to \mathrm{IT}_E(A)]$.

It turns out that one can systematically transform the code for defining $\alpha \in \mathrm{IT}_E(A)$ into a definition of $\alpha' \in \mathrm{IT}_F(A)$ when the inclusion $E \rightarrowtail F$ is given. Such an approach is not modular since one often cannot foresee the larger effect signature F when programming in $\mathrm{IT}_E(A)$. To recover modularity, we work with a general unknown F such that $E \rightarrowtail F$. This amounts to transforming all definitions of type $\mathrm{IT}_E(A)$ into dependent functions of type $\forall F.(E \rightarrowtail F) \to \mathrm{IT}_F(A)$. To see why this small change enables modular composition, consider the following example.

Given the following definitions:

$$\begin{aligned} \alpha &: \forall F.\ (E_1 \rightarrowtail F) \to \mathrm{IT}_F(A) \\ \beta &: \forall F.\ (E_2 \rightarrowtail F) \to \mathrm{IT}_F(A) \\ R &: \forall F.\ (E_3 \rightarrowtail F) \to \mathrm{IT}_F(A) \to \mathrm{IT}_F(A) \to \mathrm{IT}_F(A) \end{aligned}$$

Let $F_0 = E_1 + (E_2 + E_3)$. We can specialize α, β, R to obtain the following concrete GItrees:

$$\begin{aligned} \alpha\langle F_0, \mathrm{inl}\rangle &: \mathrm{IT}_{F_0}(A) \\ \beta\langle F_0, \mathrm{inr} \circ \mathrm{inl}\rangle &: \mathrm{IT}_{F_0}(A) \\ R\langle F_0, \mathrm{inr} \circ \mathrm{inr}\rangle &: \mathrm{IT}_{F_0}(A) \to \mathrm{IT}_{F_0}(A) \to \mathrm{IT}_{F_0}(A) \end{aligned}$$

This allows us to apply α and β to R. We do not have to restrict ourselves to the concrete signature F_0. For every F such that $\epsilon : F_0 \rightarrowtail F$, we can carry out the following specialization:

$$\begin{aligned} \alpha\langle F, \epsilon \circ \mathrm{inl}\rangle &: \mathrm{IT}_F(A) \\ \beta\langle F, \epsilon \circ \mathrm{inr} \circ \mathrm{inl}\rangle &: \mathrm{IT}_F(A) \\ R\langle F, \epsilon \circ \mathrm{inr} \circ \mathrm{inr}\rangle &: \mathrm{IT}_F(A) \to \mathrm{IT}_F(A) \to \mathrm{IT}_F(A) \end{aligned}$$

Effectively, we have defined:

$$\alpha' : \forall F.\ (F_0 \rightarrowtail F) \to \mathrm{IT}_F(A)$$
$$\beta' : \forall F.\ (F_0 \rightarrowtail F) \to \mathrm{IT}_F(A)$$
$$R' : \forall F.\ (F_0 \rightarrowtail F) \to \mathrm{IT}_F(A) \to \mathrm{IT}_F(A) \to \mathrm{IT}_F(A)$$

3 Case Study: pseudo-randomness as an effect

We demonstrate how to model pseudo-randomness in GItrees in a straight-forward approach, upon which we develop a denotational semantics for a programming languages with built-in support for pseudo-random number generators (PRNGs).

A PRNG is a stateful deterministic algorithm for creating a stream of seemingly random data from a seed. Formally, a PRNG is a tuple (S, T, f, g), where S is the state space, T is the output space, $f : S \to S$ is the state update function and $g : S \to T$ is the read out function. A generator with seed $s_0 \in S$ generates the sequence $t_n = g(s_n)$ on the n-th invocation, where $s_n = f(s_{n-1})$. Compared to true randomness, which is typically implemented by measuring physically unpredictable phenomenon, pseudo-randomness is considerably easier to implement, less costly, and more robust to the physical environment. More importantly, true randomness have quite limited throughput due to the time needed for performing measurements. PRNG does not create such limit. In practice, PRNG are often used to implement to efficient randomized algorithms and tend to have satisfying performance in non-adversarial settings.

The section is organized in a step-by-step manner in the hope that it can be read as a recipe for readers who want to model an effect that is not already available in the GItree formalization or build a denotational semantics for studying an effectful language.

3.1 A language with the PRNG effect

We start by defining a programming language featuring the PRNG effect λ_{prng}. For simplicity, the state space and output space of generators in λ_{prng} are natural numbers and the semantics of λ_{prng} is parameterized over a PRNG $(\mathbb{N}, \mathbb{N}, \text{update}, \text{read})$.

Definition 3.1 (λ_{prng} raw syntax)**.**

$$\begin{array}{rrl}
\text{Types} & \tau & ::= \mathsf{unit} \mid \mathsf{nat} \mid \mathsf{prng} \mid \tau \to \tau \\
\text{Values} & v & ::= \langle\rangle \mid n \mid \ell \mid \mathsf{rec}\ f(x) = e \\
\text{Exprs} & e & ::= v \mid x \mid e_1\ e_2 \mid e_1 \oplus e_2 \mid \mathsf{if}\ e_0\ \mathsf{then}\ e_1\ \mathsf{else}\ e_2 \\
 & & \quad \mid \mathbf{NewPrng} \mid \mathbf{DelPrng}\ e \mid \mathbf{Seed}\ e_1\ e_2 \mid \mathbf{Rand}\ e \\
\text{Ctxs} & K & ::= [\bullet] \mid K\ e \mid v\ K \mid K \oplus e \mid v \oplus K \mid \mathsf{if}\ K\ \mathsf{then}\ e_1\ \mathsf{else}\ e_2 \\
 & & \quad \mid \mathbf{DelPrng}\ K \mid \mathbf{Seed}\ K\ e \mid \mathbf{Seed}\ v\ K \mid \mathbf{Rand}\ K
\end{array}$$

where $n \in \mathsf{nat}$, $\ell \in \mathrm{Loc}$ and $\oplus \in \{+, -, *\}$.

Definition 3.2 (λ_{prng} small-step operational semantics)**.** The PRNG state $\sigma \in \mathrm{Loc} \xrightarrow{\text{fin}} \mathbb{N}$ is a finite map from locations to natural numbers, keeping track of the current PRNG internal state value. A λ_{prng} *program configuration* consists of the remaining program to be executed and the current state.

Pure head reductions:

$$\frac{}{((\mathsf{rec}\ f(x) = e)\ v, \sigma) \mapsto (e[(\mathsf{rec}\ f(x) = e)/f,\ v/x], \sigma)} \qquad \frac{n > 0}{(\mathsf{if}\ n\ \mathsf{then}\ e_1\ \mathsf{else}\ e_2, \sigma) \mapsto (e_1, \sigma)}$$

$$\frac{}{(\mathsf{if}\ 0\ \mathsf{then}\ e_1\ \mathsf{else}\ e_2, \sigma) \mapsto (e_2, \sigma)} \qquad \frac{v_1 \oplus v_2 = v_3}{(v_1 \oplus v_2, \sigma) \mapsto (v_3, \sigma)}$$

PRNG-related head reductions:

$$\frac{\ell \notin \mathrm{dom}(\sigma)}{(\mathbf{NewPrng}, \sigma) \mapsto (\ell,\ \sigma[\ell := 0])} \qquad \frac{\sigma(\ell) = n}{(\mathbf{Rand}\ \ell, \sigma) \mapsto (\mathrm{read}(n),\ \sigma[\ell := \mathrm{update}(n)])}$$

$$\frac{\sigma(\ell) = n}{(\mathbf{Seed}\ \ell\ m, \sigma) \mapsto (\langle\rangle,\ \sigma[\ell := m])} \qquad \frac{}{(\mathbf{DelPrng}\ \ell, \sigma) \mapsto (\langle\rangle,\ \sigma \setminus \ell)}$$

Lifting to arbitrary redexes with context-filling reduction rule

$$\frac{(e, \sigma) \mapsto (e', \sigma')}{(K[e], \sigma) \mapsto (K[e'], \sigma')}$$

We note that generator values are first-class citizens of the λ_{prng} language, which can be freely copied, discarded, passed as arguments or returned from functions. Indeed, a generator is duplicated when passed to a function that uses its argument multiple times. On top of this, copies of a generator value share the underlying generator state, making it impossible to predict the stream to be generated by a generator by observing its copies. More specifically, suppose that g' is created by duplicating g and **Rand** g gives n, the expression **Rand** g' is not guaranteed to return n.

Definition 3.3 (λ_{prng} type system)**.** For the pure part:

$$\frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \frac{}{\Gamma \vdash \langle\rangle : \mathsf{unit}} \qquad \frac{}{\Gamma \vdash n : \mathsf{nat}} \qquad \frac{\Gamma, f : \tau_1 \to \tau_2,\ x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \mathsf{rec}\ f(x) = e : \tau_1 \to \tau_2}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2} \qquad \frac{\Gamma \vdash e_1 : \mathsf{nat} \qquad \Gamma \vdash e_2 : \mathsf{nat}}{\Gamma \vdash e_1 \oplus e_2 : \mathsf{nat}} \qquad \frac{\Gamma \vdash e_0 : \mathsf{nat} \qquad \Gamma \vdash e_1 : \tau \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash \mathsf{if}\ e_0\ \mathsf{then}\ e_1\ \mathsf{else}\ e_2 : \tau}$$

For the effectful part:

$$\frac{}{\Gamma \vdash \textbf{NewPrng} : \mathsf{prng}} \qquad \frac{\Gamma \vdash e : \mathsf{prng}}{\Gamma \vdash \textbf{Rand}\ e : \mathsf{nat}} \qquad \frac{\Gamma \vdash e_l : \mathsf{prng} \qquad \Gamma \vdash e_s : \mathsf{nat}}{\Gamma \vdash \textbf{Seed}\ e_l\ e_s : \mathsf{unit}}$$

The typing rules are mostly standard and upholds the weakening and substitution lemma. We note that no typing rule is given for **DelPrng** – deleting a potentially shared generator is unsound, so the constructor remains unused in well-typed programs. While it is possible to give a proper typing rule for **DelPrng** by adopting a linear type discpline, we would like to not overcomplicate λ_{prng}.

3.2 Modeling the PRNG effect in guarded interaction trees

Here, we follow the recipe in by Stepanenko et al. [Ste+25] to model the PRNG effects.

Step 1: Identify the Key Operations. The PRNG effect involves three key operations `new`, `delete`, `seed`, and `rand`.

- Creation of a generator: Every **NewPrng** call create a new generator, for which we have to keep track of its internal state.
- Deletion of a generator: When **DelPrng** g is called, we have to find the generator corresponding to the value g and remove it.
- Seeding of a generator: When **Seed** g s is executed, we have to find the generator corresponding to the value g and update its internal state.
- Generation of a random number: Each time **Rand** g is executed, we have to find the underlying generator, read out a number, and update its state.

Step 2: Design the State. To implement the PRNG effect, we maintain the internal state of all generator created so far. Furthermore, each generator should have a unique identifier to enable efficient lookup and sharing. We use the `gmap` data structure to store a finite map from locations to natural numbers as the state of the PRNG effect, and define the state functor as $State(X) \triangleq \text{Loc} \xrightarrow{\text{fin}} \mathbb{N}$.

Step 3: Define the Effect Signatures.

- Effects: $E = (\text{new}, \text{del}, \text{seed}, \text{rand})$
- Creation effect: $\mathsf{Ins}_{\text{new}}(X) = \mathbf{1}$ and $\mathsf{Outs}_{\text{new}}(X) = \text{Loc}$. To create a new generator, no data have to be provided. A location, where the generator state is kept, will be returned.
- Deletion effect: $\mathsf{Ins}_{\text{del}}(X) = \text{Loc}$ and $\mathsf{Outs}_{\text{del}} = \mathbf{1}$. To delete a generator, the location of the generator state has to be provided. No meaningful data will be returned.
- Seeding effect: $\mathsf{Ins}_{\text{seed}}(X) = \text{Loc} \times \mathbb{N}$ and $\mathsf{Outs}_{\text{seed}}(X) = \mathbf{1}$. To seed a generator, the state location and the new random seed have to be provided. No meaningful data will be returned.

- Generation effect: $\mathsf{Ins}_{\mathrm{rand}}(X) = \mathrm{Loc}$ and $\mathsf{Outs}_{\mathrm{rand}}(X) = \mathbb{N}$. To get a random number, location to a generator state has to be provided. A number read out from the random state will be returned.

Step 4: Implement the Reifiers. The reification of the PRNG effects is straight-forward. Recall that the reifier r_i of an effect $i \in E$ ought to be a function $\mathsf{Ins}_i(X) \times \mathit{State}(X) \to (\mathsf{Outs}_i(X) \times \mathit{State}(X))^?$

$$
\begin{aligned}
r_{\mathrm{new}}((), \sigma) &\triangleq \mathrm{Some}(l, \sigma[l \mapsto 0])\ l \notin \mathrm{dom}(\sigma) \text{ is a fresh location} \\
r_{\mathrm{del}}(l, \sigma) &\triangleq \begin{cases} \mathrm{Some}((), \mathrm{delete}(\sigma, l)) & l \in \mathrm{dom}(\sigma) \\ \mathrm{None} & l \notin \mathrm{dom}(\sigma) \end{cases} \\
r_{\mathrm{seed}}((l, s), \sigma) &\triangleq \begin{cases} \mathrm{Some}((), \sigma[l \mapsto s]) & l \in \mathrm{dom}(\sigma) \\ \mathrm{None} & l \notin \mathrm{dom}(\sigma) \end{cases} \\
r_{\mathrm{rand}}(l, \sigma) &\triangleq \begin{cases} \mathrm{Some}(\mathrm{read}(n), \sigma[l \mapsto \mathrm{update}(n)]) & l \in \mathrm{dom}(\sigma) \wedge \sigma(l) = n \\ \mathrm{None} & l \notin \mathrm{dom}(\sigma) \end{cases}
\end{aligned}
$$

Step 5: Define Convenient Wrapper Functions. A GItree can call an effects by using the `Vis` smart constructor. However, directly working with such a low-level construction is cumbersome and error prone. To make our programming and reasoning about the PRNG effect easier, we provide some wrappers around the Vis constructor:

- $\mathrm{PRNG\_NEW}(k : \mathrm{Loc} \to X) \triangleq \mathrm{Vis}_{\mathrm{new}}((), \mathrm{next} \circ k)$
- $\mathrm{PRNG\_DEL\_k}(l : \mathrm{Loc}, k : \mathbf{1} \to X) \triangleq \mathrm{Vis}_{\mathrm{del}}(l, \mathrm{next} \circ k)$ and $\mathrm{PRNG\_DEL}(l) \triangleq \mathrm{PRNG\_DEL\_k}(l, \mathrm{Ret}())$
- $\mathrm{PRNG\_SEED\_k}(l : \mathrm{Loc}, n : \mathbb{N}, k : \mathbf{1} \to X) \triangleq \mathrm{Vis}_{\mathrm{seed}}((l, n), \mathrm{next} \circ k)$ and $\mathrm{PRNG\_SEED}(l, n) \triangleq \mathrm{PRNG\_SEED\_k}(l, n, \mathrm{Ret}())$
- $\mathrm{PRNG\_GEN\_k}(l : \mathrm{Loc}, k : \mathbb{N} \to X) \triangleq \mathrm{Vis}_{\mathrm{rand}}(l, \mathrm{next} \circ k)$ and $\mathrm{PRNG\_GEN}(l) \triangleq \mathrm{PRNG\_GEN\_k}(l, \mathrm{Ret})$

Equipped with these high-level programming support, one can readily encode randomized algorithms in GItrees. We give a simple Las Vegas algorithm GItree program, which uses the PRNG effect and the higher-order store effect:

$$
\begin{aligned}
\mathrm{neg\_bool}(\alpha_n) &\triangleq \mathrm{NatOp}_{\mathrm{sub}}(\mathrm{Ret}(1))(\alpha_n) \\
\mathsf{las\_vegas}(sd, init, test) &\triangleq \mathrm{PRNG\_NEW}\ (\lambda r. \\
&\qquad \mathrm{ALLOC}\ (\mathrm{Ret}(init))\ (\lambda s. \\
&\qquad\quad \mathrm{PRNG\_SEED}\ r\ sd; \\
&\qquad\quad \mathrm{While}\ (\mathrm{neg\_bool}\ (test \bullet_{\mathrm{cbv}} \mathrm{READ}\ s)) \\
&\qquad\qquad (\mathrm{LET}\ x = \mathrm{PRNG\_GEN}\ r\ \mathrm{in}\ \mathrm{WRITE}\ s\ x) \\
&\qquad\quad \mathrm{READ}\ s))
\end{aligned}
$$

The behavior of the program can be described as:

1. Take an initial guess $init \in \mathbb{N}$, a seed $sd \in \mathbb{N}$, and a test predicate $test : \mathbb{N} \to \mathbf{2}$ encoded as a GItree.
2. Create a generator r with initial seeds sd.
3. Allocate a store cell s which holds $init$ initially.
4. If the test fails on the current value s, sample a fresh random number using r, stores it in s, and retry.
5. When s passes the test, the loop stops.
6. Read a from s and return the value.

Intuitively, this program should be safe to run and may terminate with a value v such that $test(v) = \mathbf{T}$.

Step 6: Derive Program Logic Rules. While the reifier endows a semantics for the PRNG effect, it is hard to directly prove properties of GItrees involving the PRNG effect. We will derive some weakest precondition proof rules to facilitate modular reasoning of PRNG effects.

As we have suggested before, we need to create a ghost theory to enable reasoning about effect reification with partial view on the state. We start by pondering the right notion of a partial view for our purpose. Given the state functor definition $\mathit{State}(X) \triangleq \mathrm{Loc} \xrightarrow{\mathrm{fin}} \mathbb{N}$, it is natural to reuse the "points-to" predicate $\ell \hookrightarrow n$, which gives exclusive ownership of the store cell at location l and the knowledge that the location is valid. However, we quickly realize that this notion of partial view is too strong for the following two reasons.

1. Actual generator states should not be exposed: We typically expect to prove properties of programs with PRNG effects without knowing the exact generator state and the pseudo-random sequence to be generated. This is because PRNG is often used as a practical alternative to true randomness and non-determinism. Take the Las Vegas GItree as an example, the proper correctness specification of that program should not involve the state of generators.

2. The partial view Iris proposition should be a duplicable resource: We will eventually interpret λ_{prng} programs as GItrees with PRNG effects. In the programming language, copies of a generator values of type prng can be freely created and discarded, and it should be always safe to use a copy.

Motivated by these analysis, we weaken the "points-to" predicate $\ell \hookrightarrow n$ to the "valid generator" predicate $\mathsf{known\_prng}(\ell)$ to represent the knowledge that a valid generator state is being stored at the location ℓ. Before we present the resource algebra constructions to realize the ghost theory, let's reflect on what reasoning principles our ghost theory should support.

$$\frac{\ell : \mathrm{Loc}}{\mathsf{known\_prng}(\ell) : iProp} \qquad \frac{\sigma : \mathrm{Loc} \xrightarrow{\text{fin}} \mathbb{N}}{\mathsf{has\_prngs}(\sigma) : iProp}$$

GH-PARTIAL-DUP
$$\frac{}{\mathsf{known\_prng}(\ell) \vdash \mathsf{known\_prng}(\ell) * \mathsf{known\_prng}(\ell)}$$

GH-FULL-NODUP
$$\frac{}{\mathsf{has\_prngs}(\sigma) * \mathsf{has\_prngs}(\sigma) \vdash \mathsf{False}}$$

GH-STORE-ALLOC
$$\frac{\sigma : \mathrm{Loc} \xrightarrow{\text{fin}} \mathbb{N}}{\vdash \Rightarrow \mathsf{has\_prngs}(\sigma)}$$

GH-LOC-ALLOC
$$\frac{\ell \notin \mathrm{dom}(\sigma) \qquad n \in \mathbb{N}}{\mathsf{has\_prngs}(\sigma) \vdash \Rightarrow (\mathsf{has\_prngs}(\sigma[l := n]) * \mathsf{known\_prng}(l))}$$

GH-GET
$$\frac{}{\mathsf{has\_prngs}(\sigma) * \mathsf{known\_prng}(\ell) \vdash \mathsf{has\_prngs}(\sigma) * \exists n \in \mathbb{N} \,.\, \sigma(\ell) = n}$$

GH-SET
$$\frac{n \in \mathbb{N}}{\mathsf{has\_prngs}(\sigma) * \mathsf{known\_prng}(\ell) \vdash \Rightarrow \mathsf{has\_prngs}(\sigma[l := n])}$$

To implement this ghost theory, we use an exclusive resource algebra $\mathsf{exclR}(\mathsf{gmapUR}(\mathrm{Loc}, \mathbb{N}))$ to track the actual store and an authoritative resource algebra $\mathsf{authR}(\mathsf{gsetR}(\mathrm{Loc}))$ to track the set of valid locations in the store. Let γ_K and γ_V be two fresh ghost names, we define

$$\mathsf{has\_prngs}(\sigma) \triangleq \boxed{\mathrm{Excl}(\sigma)}^{\gamma_V} * \boxed{\bullet\ \mathrm{dom}(\sigma)}^{\gamma_K} \qquad \mathsf{known\_prng}(l) \triangleq \boxed{\circ\ (\{l\})}^{\gamma_K}$$

All the proof rules can be justified by using the properties of the constituent CMRAs. We note that our construction makes the partial view resource $\mathsf{known\_prng}(\ell)$ persistent and hence duplicable.

Building on top of this ghost theory, program logic capturing the intuition of safety of PRNG effects can be derived from the WP-REIFY rule.

WP-PRNG-NEW
$$\frac{\boxed{\exists \sigma.\ \mathsf{has\_state}(\sigma) * \mathsf{has\_prngs}(\sigma)}^{\iota p} \qquad \triangleright\triangleright (\forall \ell.\ \mathsf{known\_prng}(\ell) \mathrel{-\!\!*} \mathsf{wp}\ k\ \ell\ \{\Phi\})}{\mathsf{wp}\ \mathrm{PRNG\_NEW}\ k\ \{\Phi\}}$$

WP-PRNG-GEN
$$\frac{\boxed{\exists \sigma.\ \mathsf{has\_state}(\sigma) * \mathsf{has\_prngs}(\sigma)}^{\iota p} \qquad \triangleright\, \mathsf{known\_prng}(\ell) \qquad \triangleright\triangleright \forall n.\ \Phi(\mathrm{Ret}(n))}{\mathsf{wp}\ \mathrm{PRNG\_GEN}\ \ell\ \{\Phi\}}$$

WP-PRNG-SEED
$$\frac{\boxed{\exists \sigma.\ \mathsf{has\_state}(\sigma) * \mathsf{has\_prngs}(\sigma)}^{\iota p} \qquad \triangleright\, \mathsf{known\_prng}(\ell) \qquad \triangleright\triangleright \Phi(\mathrm{Ret}(()))}{\mathsf{wp}\ \mathrm{PRNG\_SEED}\ \ell\ m\ \{\Phi\}}$$

The WP-PRNG-NEW rule says that PRNG_NEW k is safe if the k can continue safely with a location ℓ regardless of the actual value of ℓ and the initial state at that location. The WP-PRNG-GEN rule says that to safely execute PRNG_GEN ℓ, ℓ should be valid and no assumption of the generated number can be made. The WP-PRNG-SEED rule says that executing PRNG_SEED ℓ is always safe, provided that ℓ is a valid generator. No rule for PRNG_DEL can be derived. This is because $\mathsf{known\_prng}(\ell)$ is persistent, which implies $l \in \mathrm{dom}(\sigma)$ can never be invalidated.

Another set of rules with the explicit ownership of the full state threaded through is also provided. These rules are used to related the execution of a GItree with PRNG effect and a λ_{prng} program.

WP-PRNG-NEW'

$$\frac{l = \text{fresh}(\text{dom}(\sigma)) \qquad \mathsf{has\_state}(\sigma) \qquad \mathsf{has\_prngs}(\sigma) \qquad \triangleright\triangleright(\mathsf{has\_state}(\sigma[l := 0]) * \mathsf{has\_prngs}(\sigma[l := 0]) \mathrel{-\!\!*} \mathsf{known\_prng}(\ell) \mathrel{-\!\!*} \mathsf{wp}\ k\ \ell\ \{\Phi\})}{\mathsf{wp}\ \text{PRNG\_NEW}\ k\ \{\Phi\}}$$

WP-PRNG-GEN'

$$\frac{\mathsf{has\_state}(\sigma) \qquad \mathsf{has\_prngs}(\sigma) \qquad \mathsf{known\_prng}(\ell) \qquad \sigma(\ell) = n \qquad \triangleright\triangleright(\mathsf{has\_state}(\sigma[l := \text{update}(n)]) * \mathsf{has\_prngs}(\sigma[l := \text{update}(n)]) \mathrel{-\!\!*} \mathsf{known\_prng}(\ell) \mathrel{-\!\!*} \Phi(\text{Ret}(\text{read}(n)))}{\mathsf{wp}\ \text{PRNG\_GEN}\ \ell\ \{\Phi\}}$$

WP-PRNG-SEED'

$$\frac{\mathsf{known\_prng}(\ell) \qquad \mathsf{has\_state}(\sigma) \qquad \mathsf{has\_prngs}(\sigma) \qquad \triangleright\triangleright(\mathsf{has\_state}(\sigma[l := m]) * \mathsf{has\_prngs}(\sigma[l := m]) \mathrel{-\!\!*} \mathsf{known\_prng}(\ell) \mathrel{-\!\!*} \Phi(\text{Ret}(())))}{\mathsf{wp}\ \text{PRNG\_SEED}\ \ell\ m\ \{\Phi\}}$$

Similarly, two proof rules for the higher-order store effect can be derived.

WP-READ

$$\frac{\boxed{\exists \sigma_h.\ \mathsf{has\_state}(\sigma_h)}^{\iota h} \qquad l \hookrightarrow \alpha \qquad \triangleright(l \hookrightarrow \alpha \mathrel{-\!\!*} \mathsf{wp}\ \alpha\ \{\Phi\})}{\mathsf{wp}\ (!l)\ \{\Phi\}}$$

WP-STORE

$$\frac{\boxed{\exists \sigma_h.\ \mathsf{has\_state}(\sigma_h)}^{\iota h} \qquad l \hookrightarrow \alpha \qquad \triangleright(l \hookrightarrow \beta \mathrel{-\!\!*} \Phi(\text{Ret}(())))}{\mathsf{wp}\ (l \leftarrow \beta)\ \{\Phi\}}$$

To conclude this part, we showcase a proof that the Las Vegas GItree program always finds a valid solution upon termination.

Theorem 3.1 (Partial correctness of Las Vegas)**.** *Let $f : \mathbb{N} \to \mathbf{2}$ be a predicate, test be a GItree that implements f where*

$$\square\,(\forall n.\ \mathit{wp}\ \{\mathit{test} \bullet_{\mathit{cbv}} \text{Ret}(n)\}\ \{\beta.\ (\beta \equiv \text{Ret}(1) \land f(n) = \mathbf{T}) \lor (\beta \equiv \text{Ret}(0) \land f(n) = \mathbf{F})\})$$

That is, test $\bullet_{cbv}$ Ret(n) reduces to Ret(1) *if $f(n)$ holds and* Ret(0) *otherwise.*

For every $sd, init \in \mathbb{N}$, the following weakest precondition can be derived

$$\boxed{\exists \sigma_h.\ \mathit{has\_state}(\sigma_h)}^{\iota h} * \boxed{\exists \sigma.\ \mathit{has\_state}(\sigma) * \mathit{has\_prngs}(\sigma)}^{\iota p} \vdash \mathit{wp}\ \mathsf{las\_vegas}(sd, init, test)\ \{\beta.\ \exists n.\ \beta \equiv \text{Ret}(n) \land f(n) = \mathbf{T}\}$$

```
PRNG_NEW (λr.
    ALLOC (Ret(init)) (λs.
      PRNG_SEED r sd;
      While (neg_bool (test •cbv READ s))
        (LET x = PRNG_GEN r in WRITE s x);
      READ s))
```

Proof. The idea is to repeatedly step the GItree with symbolic execution proof rules and handle the loop with Löb induction. We should use the implicit state rules since the effect reifier states are wrapped in invariants.

1. Unfold the definition of las_vegas.

$$\begin{array}{l}\boxed{\exists \sigma_h.\ \mathsf{has\_state}(\sigma_h)}^{\iota h} * \boxed{\exists \sigma.\ \mathsf{has\_state}(\sigma) * \mathsf{has\_prngs}(\sigma)}^{\iota p} \\ \square\,(\forall n.\ \mathsf{wp}\ \{test \bullet_{\text{cbv}} \text{Ret}(n)\}\ \{\beta.\ (\beta \equiv \text{Ret}(1) \land f(n) = \mathbf{T}) \lor (\beta \equiv \text{Ret}(0) \land f(n) = \mathbf{F})\}) \\ \vdash \mathsf{wp}\ \{\text{PRNG\_NEW}\ (\lambda r.\text{ALLOC}\ (\text{Ret}(init))\ (\lambda s.\boxed{\cdots}))\}\ \{\beta.\ \exists n.\ \beta \equiv \text{Ret}(n) \land f(n) = \mathbf{T}\}\end{array}$$

2. Allocation with WP-PRNG-NEW and WP-ALLOC.

$$\begin{array}{l}\boxed{\exists \sigma_h.\ \mathsf{has\_state}(\sigma_h)}^{\iota h} * \boxed{\exists \sigma.\ \mathsf{has\_state}(\sigma) * \mathsf{has\_prngs}(\sigma)}^{\iota p} * \mathsf{known\_prng}(r) * s \hookrightarrow \text{Ret}(init) \\ \square\,(\forall n.\ \mathsf{wp}\ \{test \bullet_{\text{cbv}} \text{Ret}(n)\}\ \{\beta.\ (\beta \equiv \text{Ret}(1) \land f(n) = \mathbf{T}) \lor (\beta \equiv \text{Ret}(0) \land f(n) = \mathbf{F})\}) \\ \vdash \mathsf{wp}\ \{\text{PRNG\_SEED}\ r\ sd; \boxed{\cdots}\}\ \{\beta.\ \exists n.\ \beta \equiv \text{Ret}(n) \land f(n) = \mathbf{T}\}\end{array}$$

3. $-;\ e_2$ is a GItree homomorphisms. Apply WP-BIND.

$$\boxed{\exists \sigma_h.\ \mathsf{has\_state}(\sigma_h)}^{\iota h} * \boxed{\exists \sigma.\ \mathsf{has\_state}(\sigma) * \mathsf{has\_prngs}(\sigma)}^{\iota p} * \mathsf{known\_prng}(r) * s \hookrightarrow \mathrm{Ret}(\mathit{init})$$
$$\square\,(\forall n.\ \mathsf{wp}\,\{\mathit{test} \bullet_{\mathrm{cbv}} \mathrm{Ret}(n)\}\,\{\beta.\ (\beta \equiv \mathrm{Ret}(1) \wedge f(n) = \mathbf{T}) \vee (\beta \equiv \mathrm{Ret}(0) \wedge f(n) = \mathbf{F})\})$$
$$\vdash \mathsf{wp}\,\{\mathrm{PRNG\_SEED}\ r\ \mathit{sd}\}\,\{\_.\ \mathsf{wp}\,\boxed{\cdots}\,\{\beta.\ \exists n.\ \beta \equiv \mathrm{Ret}(n) \wedge f(n) = \mathbf{T}\}\}$$

4. Update the PRNG seed with WP-SEED.

$$\boxed{\exists \sigma_h.\ \mathsf{has\_state}(\sigma_h)}^{\iota h} * \boxed{\exists \sigma.\ \mathsf{has\_state}(\sigma) * \mathsf{has\_prngs}(\sigma)}^{\iota p} * \mathsf{known\_prng}(r) * s \hookrightarrow \mathrm{Ret}(\mathit{init})$$
$$\square\,(\forall n.\ \mathsf{wp}\,\{\mathit{test} \bullet_{\mathrm{cbv}} \mathrm{Ret}(n)\}\,\{\beta.\ (\beta \equiv \mathrm{Ret}(1) \wedge f(n) = \mathbf{T}) \vee (\beta \equiv \mathrm{Ret}(0) \wedge f(n) = \mathbf{F})\})$$
$$\vdash \mathsf{wp}\,\{\mathrm{While}\ (\mathrm{neg\_bool}\ (\mathit{test} \bullet_{\mathrm{cbv}} \mathrm{READ}\ s))\ (\boxed{\cdots}); \mathrm{READ}\ s\}\,\{\beta.\ \exists n.\ \beta \equiv \mathrm{Ret}(n) \wedge f(n) = \mathbf{T}\}$$

5. Apply the Löb rule with init generalized.

$$\boxed{\exists \sigma_h.\ \mathsf{has\_state}(\sigma_h)}^{\iota h} * \boxed{\exists \sigma.\ \mathsf{has\_state}(\sigma) * \mathsf{has\_prngs}(\sigma)}^{\iota p} * \mathsf{known\_prng}(r) * s \hookrightarrow \mathrm{Ret}(\mathit{init})$$
$$\square\,(\forall n.\ \mathsf{wp}\,\{\mathit{test} \bullet_{\mathrm{cbv}} \mathrm{Ret}(n)\}\,\{\beta.\ (\beta \equiv \mathrm{Ret}(1) \wedge f(n) = \mathbf{T}) \vee (\beta \equiv \mathrm{Ret}(0) \wedge f(n) = \mathbf{F})\})$$
$$\triangleright \forall i.\ s \hookrightarrow \mathrm{Ret}(i) \mathrel{-\!\!*} \mathsf{wp}\,\boxed{\cdots}\,\{\beta.\ \exists n.\ \beta \equiv \mathrm{Ret}(n) \wedge f(n) = \mathbf{T}\}$$
$$\vdash \mathsf{wp}\,\{\mathrm{While}\ (\mathrm{neg\_bool}\ (\mathit{test} \bullet_{\mathrm{cbv}} \mathrm{READ}\ s))\ (\boxed{\cdots}); \mathrm{READ}\ s\}\,\{\beta.\ \exists n.\ \beta \equiv \mathrm{Ret}(n) \wedge f(n) = \mathbf{T}\}$$

6. While is a guarded fixed point validating the following equation

$$\forall \alpha, \beta.\ \mathrm{While}\ \alpha\ \beta \equiv \mathrm{If}\ \alpha\ (\beta;\ \mathrm{Tick}(\mathrm{While}\ \alpha\ \beta))\ \mathrm{Ret}(())$$

Apply WP-BIND to evaluate $\mathrm{neg\_bool}\ (\mathit{test} \bullet_{\mathrm{cbv}} \mathrm{READ}\ s)$ first.

$$\boxed{\exists \sigma_h.\ \mathsf{has\_state}(\sigma_h)}^{\iota h} * \boxed{\exists \sigma.\ \mathsf{has\_state}(\sigma) * \mathsf{has\_prngs}(\sigma)}^{\iota p} * \mathsf{known\_prng}(r) * s \hookrightarrow \mathrm{Ret}(\mathit{init})$$
$$\square\,(\forall n.\ \mathsf{wp}\,\{\mathit{test} \bullet_{\mathrm{cbv}} \mathrm{Ret}(n)\}\,\{\beta.\ (\beta \equiv \mathrm{Ret}(1) \wedge f(n) = \mathbf{T}) \vee (\beta \equiv \mathrm{Ret}(0) \wedge f(n) = \mathbf{F})\})$$
$$\triangleright \forall i.\ s \hookrightarrow \mathrm{Ret}(i) \mathrel{-\!\!*} \mathsf{wp}\,\boxed{\cdots}\,\{\beta.\ \exists n.\ \beta \equiv \mathrm{Ret}(n) \wedge f(n) = \mathbf{T}\}$$
$$\vdash \mathsf{wp}\,\{\mathrm{neg\_bool}\ (\mathit{test} \bullet_{\mathrm{cbv}} \mathrm{READ}\ s)\}\,\{\gamma.\ \mathsf{wp}\ \mathrm{If}\ \gamma\ \boxed{\cdots}\ \mathrm{Ret}(())\,\{\beta.\ \exists n.\ \beta \equiv \mathrm{Ret}(n) \wedge f(n) = \mathbf{T}\}\}$$

7. Apply WP-READ to read the from the store cell $s \hookrightarrow \mathrm{Ret}(\mathit{init})$. Perform a case analysis on $f(\mathit{init})$.

8. Case $f(\mathit{init}) = \mathbf{T}$. Show that the branch condition evaluates to $\gamma = \mathrm{Ret}(0)$, using the specification of neg_bool and test. Rewrite with the following equation for If

$$\forall \alpha, \beta.\ \mathrm{If}\ (\mathrm{Ret}(0))\ \alpha\ \beta \equiv \beta$$

The remaining proof goal

$$\boxed{\exists \sigma_h.\ \mathsf{has\_state}(\sigma_h)}^{\iota h} * \boxed{\exists \sigma.\ \mathsf{has\_state}(\sigma) * \mathsf{has\_prngs}(\sigma)}^{\iota p} * \mathsf{known\_prng}(r) * s \hookrightarrow \mathrm{Ret}(\mathit{init})$$
$$\square\,(\forall n.\ \mathsf{wp}\,\{\mathit{test} \bullet_{\mathrm{cbv}} \mathrm{Ret}(n)\}\,\{\beta.\ (\beta \equiv \mathrm{Ret}(1) \wedge f(n) = \mathbf{T}) \vee (\beta \equiv \mathrm{Ret}(0) \wedge f(n) = \mathbf{F})\})$$
$$\triangleright \forall i.\ s \hookrightarrow \mathrm{Ret}(i) \mathrel{-\!\!*} \mathsf{wp}\,\boxed{\cdots}\,\{\beta.\ \exists n.\ \beta \equiv \mathrm{Ret}(n) \wedge f(n) = \mathbf{T}\}$$
$$\vdash \mathsf{wp}\ \mathrm{READ}\ s\,\{\beta.\ \exists n.\ \beta \equiv \mathrm{Ret}(n) \wedge f(n) = \mathbf{T}\}$$

Apply WP-READ to complete this case.

9. Case $f(\mathit{init}) = \mathbf{F}$. Show that the branch condition evaluates to $\gamma = \mathrm{Ret}(1)$, using the specification of neg_bool and test. Rewrite with the following equation for If

$$\forall \alpha, \beta.\ \mathrm{If}\ (\mathrm{Ret}(1))\ \alpha\ \beta \equiv \alpha$$

The remaining proof goal

$$\boxed{\exists \sigma_h.\ \mathsf{has\_state}(\sigma_h)}^{\iota h} * \boxed{\exists \sigma.\ \mathsf{has\_state}(\sigma) * \mathsf{has\_prngs}(\sigma)}^{\iota p} * \mathsf{known\_prng}(r) * s \hookrightarrow \mathrm{Ret}(\mathit{init})$$
$$\square\,(\forall n.\ \mathsf{wp}\,\{\mathit{test} \bullet_{\mathrm{cbv}} \mathrm{Ret}(n)\}\,\{\beta.\ (\beta \equiv \mathrm{Ret}(1) \wedge f(n) = \mathbf{T}) \vee (\beta \equiv \mathrm{Ret}(0) \wedge f(n) = \mathbf{F})\})$$
$$\triangleright \forall i.\ s \hookrightarrow \mathrm{Ret}(i) \mathrel{-\!\!*} \mathsf{wp}\,\boxed{\cdots}\,\{\beta.\ \exists n.\ \beta \equiv \mathrm{Ret}(n) \wedge f(n) = \mathbf{T}\}$$
$$\vdash \mathsf{wp}\,\{\mathrm{LET}\ x = \mathrm{PRNG\_GEN}\ r\ \mathrm{in\ WRITE}\ s\ x; \boxed{\cdots}\}\,\{\beta.\ \exists n.\ \beta \equiv \mathrm{Ret}(n) \wedge f(n) = \mathbf{T}\}$$

Apply WP-PRNG-GEN to generate a number, denoted by n. Apply WP-STORE to write $\mathrm{Ret}(n)$ into s. In a later step, one has to show

$$\boxed{\exists \sigma_h.\ \mathsf{has\_state}(\sigma_h)}^{\iota h} * \boxed{\exists \sigma.\ \mathsf{has\_state}(\sigma) * \mathsf{has\_prngs}(\sigma)}^{\iota p} * \mathsf{known\_prng}(r) * s \hookrightarrow \mathrm{Ret}(n)$$
$$\square\,(\forall n.\ \mathsf{wp}\,\{\mathit{test} \bullet_{\mathrm{cbv}} \mathrm{Ret}(n)\}\,\{\beta.\ (\beta \equiv \mathrm{Ret}(1) \wedge f(n) = \mathbf{T}) \vee (\beta \equiv \mathrm{Ret}(0) \wedge f(n) = \mathbf{F})\})$$
$$\forall i.\ s \hookrightarrow \mathrm{Ret}(i) \mathrel{-\!\!*} \mathsf{wp}\,\boxed{\cdots}\,\{\beta.\ \exists n.\ \beta \equiv \mathrm{Ret}(n) \wedge f(n) = \mathbf{T}\}$$
$$\vdash \mathsf{wp}\,\boxed{\cdots}\,\{\beta.\ \exists n.\ \beta \equiv \mathrm{Ret}(n) \wedge f(n) = \mathbf{T}\}$$

Now we are under at least one later modality, the induction hypothesis becomes available. The first iteration of the while loop is completed and the remaining program matches with the induction hypothesis. Apply IH with $i = n$ to complete this case.

□

3.3 Interpreting λ_{prng} in guarded interaction trees

Now that a theory of PRNG effects is in place, we can give a denotational semantics of λ_{prng} by interpreting programs as GItrees with PRNG effects. Let E be an effect signature such that $\text{prng} \rightarrowtail E$, A be a type such that $A \cong \mathbf{1} + \mathbb{N} + \text{Loc} + B$ for some B. We can define a map $[\![-]\!]_\delta$ from λ_{prng} expressions to $\text{IT}_E(A)$, where $\delta : \text{Var} \to \text{IT}$ is a substitution environment for free variables. The interpretation is depicted in the following figure and should be straight-forward.

$$
\begin{aligned}
[\![\langle\rangle]\!]_\delta &\triangleq \text{Ret}(()) \quad [\![n]\!]_\delta \triangleq \text{Ret}(n) \quad [\![\ell]\!]_\delta \triangleq \text{Ret}(\ell) \\
[\![x]\!]_\delta &\triangleq \delta(x) \\
[\![e_1 \oplus e_2]\!]_\delta &\triangleq \text{NatOp}_\oplus([\![e_1]\!]_\delta)([\![e_2]\!]_\delta) \\
[\![\mathsf{if}\ e_0\ \mathsf{then}\ e_1\ \mathsf{else}\ e_2]\!]_\delta &\triangleq \text{If}([\![e_0]\!]_\delta)([\![e_1]\!]_\delta)([\![e_2]\!]_\delta) \\
[\![e_1\ e_2]\!]_\delta &\triangleq [\![e_1]\!]_\delta \bullet_{\text{cbv}} [\![e_2]\!]_\delta \\
[\![\mathsf{rec}\ f(x) = e]\!]_\delta &\triangleq \mathit{fix}\ \lambda(\alpha : \blacktriangleright\text{IT}).\ \text{Fun}\left(\blacktriangleright \circ \left(\lambda\alpha'.\ v.\ [\![e]\!]_{\delta[x \mapsto v,\ f \mapsto \alpha']}\right)(\alpha)\right) \\
[\![\mathbf{NewPrng}]\!]_\delta &\triangleq \text{PRNG\_NEW Ret} \\
[\![\mathbf{DelPrng}\ e]\!]_\delta &\triangleq \text{get\_ret}(\text{PRNG\_DEL})([\![e]\!]_\delta) \\
[\![\mathbf{Rand}\ e]\!]_\delta &\triangleq \text{get\_ret}(\text{PRNG\_GEN})([\![e]\!]_\delta) \\
[\![\mathbf{Seed}\ e_1\ e_2]\!]_\delta &\triangleq \text{SeedGit}\ ([\![e_1]\!]_\delta)\ ([\![e_2]\!]_\delta)
\end{aligned}
$$

Here $\mathit{fix} : \blacktriangleright\text{IT} \to \text{IT}$ is the guarded fixed point combinator; $\blacktriangleright\circ$ transforms the function of type $\text{IT} \to (\text{IT} \to \text{IT})$ into $\blacktriangleright\text{IT} \to \blacktriangleright(\text{IT} \to \text{IT})$ so that $t : \blacktriangleright\text{IT}$ can be applied; and $\text{SeedGit} : \text{IT} \to \text{IT} \to \text{IT}$ is a derived GItree combinator that evaluates the two arguments from right to left before passing them to PRNG_SEED.

We also interpret λ_{prng} evaluation contexts as GItree homomorphisms $\text{IT} \to \text{IT}$.

$$
\begin{aligned}
[\![[\bullet]]\!]_\delta(\alpha) &\triangleq \alpha \\
[\![K\ e]\!]_\delta(\alpha) &\triangleq [\![K]\!]_\delta(\alpha) \bullet_{\text{cbv}} [\![e]\!]_\delta \quad [\![v\ K]\!]_\delta(\alpha) \triangleq [\![v]\!]_\delta \bullet_{\text{cbv}} [\![K]\!]_\delta(\alpha) \\
[\![K \oplus e]\!]_\delta(\alpha) &\triangleq \text{NatOp}_\oplus([\![K]\!]_\delta(\alpha))([\![e]\!]_\delta) \quad [\![v \oplus K]\!]_\delta(\alpha) \triangleq \text{NatOp}_\oplus([\![e]\!]_\delta)([\![K]\!]_\delta(\alpha)) \\
[\![\mathsf{if}\ K\ \mathsf{then}\ e_1\ \mathsf{else}\ e_2]\!](\alpha) &\triangleq \text{If}([\![K]\!]_\delta(\alpha))([\![e_1]\!]_\delta)([\![e_2]\!]_\delta) \\
[\![\mathbf{DelPrng}\ K]\!]_\delta(\alpha) &\triangleq \text{get\_ret}(\text{PRNG\_DEL})([\![K]\!]_\delta(\alpha)) \\
[\![\mathbf{Rand}\ K]\!]_\delta(\alpha) &\triangleq \text{get\_ret}(\text{PRNG\_GEN})([\![K]\!]_\delta(\alpha)) \\
[\![\mathbf{Seed}\ K\ e]\!]_\delta(\alpha) &\triangleq \text{SeedGit}\ ([\![K]\!]_\delta(\alpha))\ ([\![e]\!]_\delta) \\
[\![\mathbf{Seed}\ v\ K]\!]_\delta(\alpha) &\triangleq \text{SeedGit}\ ([\![v]\!]_\delta)\ ([\![K]\!]_\delta(\alpha))
\end{aligned}
$$

Recall that GItree homomorphisms are GItree evaluation contexts in the operational semantics of GItrees. In other word, under the denotational semantics, not only do λ_{prng} programs becomes GItree programs, λ_{prng} evaluation contexts also corresponds to GItree evaluation contexts.

We note that our interpretation is compositional. That is, given a context C, which is not necessarily an evaluation context, the following equivalence holds.

$$\forall e_1, e_2.\ [\![e_1]\!]_\delta \equiv [\![e_2]\!]_\delta \implies [\![C[e_1]]\!]_\delta \equiv [\![C[e_2]]\!]_\delta$$

Having defined a translation from λ_{prng} to GItrees, we would like to show that $[\![\cdot]\!]$ faithfully and adequately captures the operational semantics of λ_{prng}. More specifically, for a λ_{prng} program e, the execution of e should relate to the reduction of $[\![e]\!]$, which will allow us to leverage the interpretation to prove semantic properties, e.g. type safety and contextual refinement, without directly working on the operational semantics of λ_{prng}.

Our first result is the soundness of this denotational interpretation, which can be proved directly by induction.

Theorem 3.2 (Soundness of the GItree interpretation for λ_{prng})**.** *Let e_1, e_2 be two closed λ_{prng} programs, σ_1, σ_2 be two PRNG states,*

$$(e_1, \sigma_1) \mapsto^* (e_2, \sigma_2) \implies ([\![e_1]\!], \sigma_1) \rightsquigarrow^* ([\![e_2]\!], \sigma_2)$$

It states that if a program e_1 can steps to e_2 while taking the state from σ_1 to σ_2, the GItree denotation $[\![e_1]\!]$ reduces to $[\![e_2]\!]$ and changes the refiers state accordingly.

3.4 Binary logical relation for adequacy

In this part, we prove that our denotational semantics enjoys the computational adequacy respect to the operational semantics, which connects GItree reductions back to λ_{prng} reductions.

Theorem 3.3 (Adequacy of the GItree interpretation for λ_{prng})**.** *Let* $\bullet \vdash e : \mathsf{nat}$ *be a closed* λ_{prng} *program,* n *be a natural number, and* σ, σ' *be two PRNG states,*

$$([\![e]\!], \sigma) \rightsquigarrow^* (\mathrm{Ret}(n), \sigma') \implies (e, \sigma) \mapsto^* (n, \sigma')$$

As a direct corollary, we can prove contextual equivalence by demonstrating denotational equivalence.

Corollary 3.4 (Denotational equivalence implies contextual equivalence)**.** *Let* $\Gamma \vdash e_1 : \tau$ *and* $\Gamma \vdash e_2 : \tau$ *be two* λ_{prng} *terms of type* τ *under context* Γ*,*

$$(\forall (\delta : \Gamma \to \mathrm{IT}).\ [\![e_1]\!]_\delta = [\![e_2]\!]_\delta) \implies e_1 \cong_{\text{ctx}} e_2$$

where $e_1 \cong_{\text{ctx}} e_2$ *denotes standard contextual equivalence for every context* C*, which is not necessarily an evaluation context, such that* $\bullet \vdash C[e_1] : \mathsf{nat}$ *and* $\bullet \vdash C[e_2] : \mathsf{nat}$*,*

$$\forall \sigma, \sigma', n.\ (C[e_1], \sigma) \mapsto^* (n, \sigma') \iff (C[e_2], \sigma) \mapsto^* (n, \sigma')$$

While the theorem statement concerns only base type programs, its indirectly consequences propagates to higher-order function types since the evaluation of a base type program $\bullet \vdash e : \mathsf{nat}$ might involve evaluation of arbitrarily complex functions. To make this point clear, consider the following consequences of the computational adequacy theorem:

1. For every program $\bullet \vdash f_1 : \tau_1$, where $\tau_1 = \mathsf{nat} \to \mathsf{nat}$

   $$\forall m, n, \sigma, \sigma'.\ ([\![f_1]\!] \bullet_{\text{cbv}} \mathrm{Ret}(m), \sigma) \rightsquigarrow^* (\mathrm{Ret}(n), \sigma') \implies (f_1\ m, \sigma) \mapsto^* (n, \sigma')$$

   Note that for $f_1\ m$ to evaluate to n, f_1 must first reduce to a suitable head-normal form.

2. For every program $\bullet \vdash f_2 : \tau_2$, where $\tau_2 = \tau_1 \to \mathsf{nat}$

   $$\forall f_1, n, \sigma, \sigma'.\ \bullet \vdash f_1 : \tau_1 \land ([\![f_2]\!] \bullet_{\text{cbv}} [\![f_1]\!], \sigma) \rightsquigarrow^* (\mathrm{Ret}(n), \sigma') \implies (f_2\ f_1, \sigma) \mapsto^* (n, \sigma')$$

   Similarly, f_2 must reduce to a suitable head-normal form.

3. For every program $\bullet \vdash f_3 : \tau_3$, where$\tau_3 = \tau_2 \to \mathsf{nat}$ and

   $$\forall f_2, n, \sigma, \sigma'.\ \bullet \vdash f_2 : \tau_2 \land ([\![f_3]\!] \bullet_{\text{cbv}} [\![f_2]\!], \sigma) \rightsquigarrow^* (\mathrm{Ret}(n), \sigma') \implies (f_3\ f_2, \sigma) \mapsto^* (n, \sigma')$$

   Here f_3 must reduce to a suitable head-normal form.

To handle the higher-order nature of the adequacy theorem, we use the logical relations proof method. More specifically, we will define a family of binary logical relation over λ_{prng} terms and GItrees.

Definition 3.4 (Binary logical relation for adequacy)**.** We first define the value and expression relations over closed programs:

$$\begin{aligned}
\mathcal{V} &: \mathrm{Type} \to \mathrm{Rel}(\mathrm{IT}, \mathrm{Val}) \\
\mathcal{V}_{\mathsf{unit}}(\alpha_v, v) &\triangleq \alpha_v \equiv \mathrm{Ret}(()) \land v \equiv \langle\rangle \\
\mathcal{V}_{\mathsf{nat}}(\alpha_v, v) &\triangleq \exists n.\ \alpha_v \equiv \mathrm{Ret}(n) \land v \equiv n \\
\mathcal{V}_{\mathsf{prng}}(\alpha_v, v) &\triangleq \exists \ell.\ \alpha_v \equiv \mathrm{Ret}(\ell) \land v \equiv \ell * \mathsf{known_prng}(\ell) \\
\mathcal{V}_{\tau_1 \to \tau_2}(\alpha_v, v) &\triangleq \exists f.\ \alpha_v \equiv \mathrm{Fun}(f) \land \Box \forall \beta_v, u.\ \mathcal{V}_{\tau_1}(\beta_v, u) \mathrel{-\!\!*} \mathcal{E}_{\tau_2}(\mathrm{Fun}(f) \bullet_{\text{cbv}} \beta_v,\ v\ u) \\
\mathcal{E} &: \mathrm{Type} \to \mathrm{Rel}(\mathrm{IT}, \mathrm{Expr}) \\
\mathcal{E}_\tau(\alpha, e) &\triangleq \forall \sigma.\ \mathsf{has_state}(\sigma) * \mathsf{has_prngs}(\sigma) \mathrel{-\!\!*} \mathsf{wp}\ \alpha\ \{\alpha_v.\ \exists v, \sigma'.\quad (e, \sigma) \mapsto^* (v, \sigma')\ \} \\
&\qquad\qquad * \ \mathsf{has_state}(\sigma') \\
&\qquad\qquad * \ \mathsf{has_prngs}(\sigma') \\
&\qquad\qquad * \ \mathcal{V}_\tau(\alpha_v, v)
\end{aligned}$$

Define the context relation

$$\begin{aligned}
\mathcal{C} &: (\Gamma : \mathrm{Ctx}) \to \mathrm{Rel}(\Gamma \to \mathrm{IT}, \Gamma \to \mathrm{Val}) \\
\mathcal{C}_\Gamma(\delta, \gamma) &\triangleq \forall x \in \Gamma.\ \mathcal{V}_{\Gamma(x)}(\delta(x), \gamma(x))
\end{aligned}$$

Finally, define the semantic typing relation by lifting the expression relation to open terms.

$$\Gamma \models e : \tau \triangleq \forall \delta, \gamma.\ \mathcal{C}_\Gamma(\gamma, \delta) \mathrel{-\!\!*} \mathcal{E}_\tau([\![e]\!]_\delta,\ e[\gamma])$$

Using the symbolic execution weakest precondition proof rules, we can prove that every typing rule has a semantically well-typed relation version. We illustrate with several representative compatibility lemmas.

The following bind lemma is essential for compositional reasoning:

Lemma 3.5 (rel-bind). *Let $f : \mathrm{IT} \to \mathrm{IT}$ be a GItree homomorphism and K an evaluation context. For any relations Ψ, Φ:*

$$\mathcal{E}_\Psi(\alpha, e) \twoheadrightarrow \big(\forall \alpha_v, v.\ \Psi(\alpha_v, v) \twoheadrightarrow \mathcal{E}_\Phi(f(\alpha_v),\ K[v])\big) \twoheadrightarrow \mathcal{E}_\Phi(f(\alpha),\ K[e]).$$

Proof. Unfolding $\mathcal{E}$, fix a state σ. From the first premise we obtain $\mathsf{wp}\ \alpha\ \{\beta_v.\ \exists v, \sigma'.\ (e,\sigma) \mapsto^* (v,\sigma') * \cdots\}$. Apply WP-BIND (the homomorphism version of the bind rule) to f and K, then use the second premise to continue from the intermediate value. □

Lemma 3.6 (compat-NewPrng). $\Gamma \models \textbf{NewPrng} : \mathsf{prng}$

Proof. Fix γ, δ with $\mathcal{C}_\Gamma(\delta, \gamma)$ and a state σ. By definition, $[\![\textbf{NewPrng}]\!]_\delta \equiv$ PRNG_NEW Ret. Apply WP-PRNG-NEW (the implicit-state rule):

$$\frac{\boxed{\exists \sigma.\ \mathsf{has\_state}(\sigma) * \mathsf{has\_prngs}(\sigma)}^{\iota p} \qquad \triangleright\triangleright(\forall \ell.\ \mathsf{known\_prng}(\ell) \twoheadrightarrow \mathsf{wp}\ \mathrm{Ret}(\ell)\ \{\alpha_v.\ \exists v, \sigma'.\ (\textbf{NewPrng}, \sigma) \mapsto^* (v, \sigma') * \cdots\})}{\mathsf{wp}\ \mathrm{PRNG\_NEW}\ \mathrm{Ret}\ \{\alpha_v.\ \cdots\}}.$$

From the operational side, we have $(\textbf{NewPrng}, \sigma) \mapsto (\ell, \sigma[\ell := 0])$ where $\ell \notin \mathrm{dom}(\sigma)$. The ghost state update is justified by ISTATE-ALLOC, which provides $\mathsf{known\_prng}(\ell)$ and the updated PRNG store. This matches the value relation $\mathcal{V}_{\mathsf{prng}}(\mathrm{Ret}(\ell), \ell)$. □

Lemma 3.7 (compat-Rand). $\Gamma \models e : \mathsf{prng} \implies \Gamma \models \textbf{Rand}\ e : \mathsf{nat}$

Proof. Let $[\![e]\!]_\delta = \alpha$ and $e[\gamma] = e'$. By the induction hypothesis, $\mathcal{E}_{\mathsf{prng}}(\alpha, e')$ holds. Apply REL-BIND with $f \triangleq \mathrm{get\_ret}(\mathrm{PRNG\_GEN})$ and $K \triangleq \textbf{Rand}\ [\bullet]$:

$$\mathcal{V}_{\mathsf{prng}}(\alpha_v, \ell) \twoheadrightarrow \mathcal{E}_{\mathsf{nat}}(\mathrm{PRNG\_GEN}\ \ell,\ \textbf{Rand}\ \ell[\gamma]).$$

From $\mathcal{V}_{\mathsf{prng}}$ we have $\alpha_v \equiv \mathrm{Ret}(\ell)$, ℓ is a literal, and $\mathsf{known\_prng}(\ell)$. Applying WP-PRNG-GEN with $\boxed{\exists \sigma.\ \mathsf{has\_state}(\sigma) * \mathsf{has\_prng}}$ and the persistent $\mathsf{known\_prng}(\ell)$:

$$\mathsf{wp}\ \mathrm{PRNG\_GEN}\ \ell\ \{\beta_v.\ \exists n.\ \beta_v \equiv \mathrm{Ret}(n)\}.$$

Operationally, $(\textbf{Rand}\ \ell) \mapsto (\mathrm{read}(m))$ where $\sigma(\ell) = m$. So $\exists n.\ \beta_v \equiv \mathrm{Ret}(n)$ matches $\mathcal{V}_{\mathsf{nat}}(\beta_v, \mathrm{read}(m))$. □

Lemma 3.8 (compat-Seed). $\Gamma \models e_l : \mathsf{prng} \implies \Gamma \models e_s : \mathsf{nat} \implies \Gamma \models \textbf{Seed}\ e_l\ e_s : \mathsf{unit}$

Proof. Apply REL-BIND twice. First, with $f_1 \triangleq \mathrm{SeedGitCtxL}([\![e_s]\!])$ and $K_1 \triangleq \textbf{Seed}\ [\bullet]\ e_s$ to evaluate e_l. This yields $\mathcal{V}_{\mathsf{prng}}(\alpha_l, \ell)$ with $\mathsf{known\_prng}(\ell)$. Then, with $f_2 \triangleq \mathrm{SeedGitCtxS}(\mathrm{Ret}(\ell))$ and $K_2 \triangleq \textbf{Seed}\ \ell\ [\bullet]$ to evaluate e_s. This yields $\mathcal{V}_{\mathsf{nat}}(\alpha_s, m)$. Using SEEDGIT-RET we rewrite $\mathrm{SeedGitCtxS}(\mathrm{Ret}(\ell))(\mathrm{Ret}(m)) \equiv \mathrm{PRNG\_SEED}\ \ell\ m$. Applying WP-PRNG-SEED, we obtain safety, and the operational side is discharged by $(\textbf{Seed}\ \ell\ m) \mapsto (\langle\rangle)$. □

Lemma 3.9 (compat-App). $\Gamma \models e_1 : \tau_1 \to \tau_2 \implies \Gamma \models e_2 : \tau_1 \implies \Gamma \models e_1\ e_2 : \tau_2$

Proof. Let $[\![e_i]\!]_\delta = \alpha_i$ and $e_i[\gamma] = e_i'$. Applying REL-BIND with $f \triangleq \alpha_1 \bullet_{\mathrm{cbv}} -$ and $K \triangleq e_1' \bullet_{\mathrm{cbv}} -$ to evaluate e_2, we obtain $\mathcal{V}_{\tau_1}(\beta_v, u)$. Then apply REL-BIND again with $f \triangleq - \bullet_{\mathrm{cbv}} \beta_v$ and $K \triangleq - \bullet_{\mathrm{cbv}} u$ to evaluate e_1, obtaining $\mathcal{V}_{\tau_1 \to \tau_2}(\varphi_v, f)$. By the definition of the function value relation:

$$\mathcal{V}_{\tau_1 \to \tau_2}(\varphi_v, f) \equiv \exists \varphi'.\ \varphi_v \equiv \mathrm{Fun}(\varphi') \wedge \Box\forall \beta_v, u.\ \mathcal{V}_{\tau_1}(\beta_v, u) \twoheadrightarrow \mathcal{E}_{\tau_2}(\mathrm{Fun}(\varphi') \bullet_{\mathrm{cbv}} \beta_v,\ f\ u).$$

Instantiating with β_v, u from the first application, we get $\mathcal{E}_{\tau_2}([\![e_1\ e_2]\!]_\delta,\ (e_1\ e_2)[\gamma])$. □

Lemma 3.10 (compat-Rec). $\Gamma, f : \tau_1 \to \tau_2,\ x : \tau_1 \models e : \tau_2 \implies \Gamma \models \mathsf{rec}\ f(x) = e : \tau_1 \to \tau_2$

Proof. The interpretation gives $[\![\mathsf{rec}\ f(x) = e]\!]_\delta \equiv \mathrm{Fun}(\mathrm{next}(\lambda v.\ [\![e]\!]_{\delta[x \mapsto v; f \mapsto [\![\mathsf{rec}\ f(x) = e]\!]_\delta}))$. We apply the value case of the expression relation, which reduces to showing the function value relation:

$$\Box\forall \beta_v, u.\ \mathcal{V}_{\tau_1}(\beta_v, u) \twoheadrightarrow \mathcal{E}_{\tau_2}(\mathrm{Fun}(\mathrm{next}\ f) \bullet_{\mathrm{cbv}} \beta_v,\ (\mathsf{rec}\ f(x) = e)[\gamma]\ u).$$

Using the CBV application rule and the operational beta-reduction, we apply REL-PURE-EXPANSION:

$$\mathrm{Fun}(\mathrm{next}\ f) \bullet_{\mathrm{cbv}} \mathrm{Ret}(v) \equiv \mathrm{Tick}(f\ v), \qquad (\mathsf{rec}\ f(x) = e)\ u \mapsto e[\mathsf{rec}\ f(x) = e/f,\ u/x].$$

The goal becomes $\mathcal{E}_{\tau_2}([\![e]\!]_{\delta[f \mapsto \mathrm{Fun}(\mathrm{next}\ f),\ x \mapsto v]},\ e[\gamma, f \mapsto \mathsf{rec}\ f(x) = e[\gamma],\ x \mapsto u])$, which is justified by the induction hypothesis and a tick step. The Löb axiom internalised in Iris discharges the guardedness: the recursive occurrence of $\mathsf{rec}\ f(x) = e$ is under a $\triangleright$ modality, which is eliminated by the tick step during application. □

These lemmas are proved by mutual induction on the typing derivation. Each case uses the appropriate WP proof rule for the PRNG effect (WP-PRNG-NEW, WP-PRNG-GEN, WP-PRNG-SEED), the bind lemma REL-BIND, and the GItree equational theory. The full Coq mechanization is available in `logrel.v`.

As a direct result, we have the following theorem:

Theorem 3.11 (fundamental theorem)**.** *Every well-typed term e is related to its GItree denotation $[\![e]\!]$:*

$$\Gamma \vdash e : \tau \implies \Gamma \models e : \tau$$

The computational adequacy theorem is derived by specializing the fundamental theorem with $\Gamma = \bullet$ and $\tau = \mathsf{nat}$.

3.5 An incomplete proof for type safety

As hinted at the beginning of this section, we would like to investigate semantic properties of λ_{prng} by working at the level of GItree denotational semantics, circumventing the operational semantics of the language. We will showcase this approach with a partial proof of the type safety of λ_{prng}. As we will see, the proof cannot be completed, exposing a limination of the GItree program logic.

Let $\bullet \vdash e : \tau$ be a well-typed closed program. We apply the fundamental lemma to show $\mathcal{E}_\tau([\![e]\!], e)$, which expands to the following proposition:

$$\begin{aligned}\forall \sigma.\ \mathsf{has\_state}(\sigma) * \mathsf{has\_prngs}(\sigma) \mathrel{-\!\!*} \mathsf{wp}\ [\![e]\!]\ \{\alpha_v.\ \exists v, \sigma'. \quad & (e,\sigma) \mapsto^* (v,\sigma')\ \} \\ & * \ \mathsf{has\_state}(\sigma') \\ & * \ \mathsf{has\_prngs}(\sigma') \\ & * \ \mathcal{V}_\tau(\alpha_v, v)\end{aligned}$$

We can instantiate σ with an empty store and allocate the ghost recourse for $\mathsf{has\_state}(\sigma)$ and $\mathsf{has\_prngs}(\sigma)$.

$$\begin{aligned}\mathsf{wp}\ [\![e]\!]\ \{\alpha_v.\ \exists v, \sigma'. \quad & (e,\emptyset) \mapsto^* (v,\sigma')\ \} \\ & * \ \mathsf{has\_state}(\sigma') \\ & * \ \mathsf{has\_prngs}(\sigma') \\ & * \ \mathcal{V}_\tau(\alpha_v, v)\end{aligned}$$

We use the consequence rule of the weakest precondition to remove the ghost resources in the post-condition.

$$\mathsf{wp}\ [\![e]\!]\ \{\alpha_v.\ \exists v, \sigma'.\ (e,\emptyset) \mapsto^* (v,\sigma')\}$$

We have derived the weakest precondition for $[\![e]\!]$ with a pure post-condition. By invoking the adequacy of the GItree program logic, we conclude that $[\![e]\!]$ runs safely and reduces to a suitable GItree head normal forms if it terminates. The result immediately gives that "e cannot go wrong" safety, i.e., $(e,\emptyset)$ cannot run into an erroneous state. Suppose that the program reduces to an error, then by the soundness lemma, $[\![e]\!]$ should also run to an error, which contradicts the weakest precondition proposition.

However, we find ourselves stuck when trying to derive the progressiveness of e. This is because that all the information about the execution of e is in the post-condition, which we only get access to it when $[\![e]\!]$ reduces to a value, yet $\mathsf{wp}\ [\![e]\!]\ \{\cdot\}$ only guarantees the progressiveness of $[\![e]\!]$ with respect to $\rightsquigarrow$. Indeed, in the presence of general recursion, we have no hope of proving the termination of $[\![e]\!]$ from the weakest precondition.

To complete the proof, we would need the soundness of the GItree program logic, specialized to the denotation of λ_{prng} programs, with respect to the operational semantics of λ_{prng}. This is the kind of adequacy theorems discussed in the *program logics à la carte* paper [VSJ25]. The research paper proposed a modular approach of building program logic for effectful languages. In their settings, they define $\mathsf{wp}\ e\ \{\Phi\}$ as $\mathsf{wp}\ [\![e]\!]\ \{\Phi\}$ where $[\![\cdot]\!]$ interprets programs as interaction trees [Xia+19]. By doing so, the equational theory of ITrees and various effect theories can be reused to build program logic for different languages. Whenever a program logic is defined, an adequacy theorem linking $\mathsf{wp}\ [\![e]\!]\ \{\Phi\}$ with the operational semantics of e has to be established. To the best of our knowledge, no similar experimentation has been done for guarded interaction trees.

4 Discussion and Future Works

In this project, we explored the application of guarded interaction trees as a semantic domain for effectful higher-order languages. A GItree effect theory for pseudo-randomness and a computationally adequate denotational semantics for λ_{prng}, a functional language equipped with PRNG effects, are developed. While being successful overall, the project has several difficulties that are not satisfyingly addressed and some potentials that are not completely examined.

4.1 Modeling richer interactions

Looking forward, the GItree framework provides a promising substrate for reasoning about cyber-physical systems. In the future, we would like to extend the framework to enable modeling rich interaction between programmed controller and physical devices, in the presence of real-time constraints, non-deterministic responses, and probabilistic reactions. Physical devices are essentially random number generators whose behaviors can be approximated and partially predicted by automata models.

The first step towards this goal is to give alternative reification to the PRNG effect signature. One can observe that the signature itself is sufficient to model deterministic (pseudo-random), stochastic, and non-determinism processes. Therefore, it should be possible to reify the effect as non-deterministic choice or sampling from a discrete probabilistic distribution. Depending on how the effect is how the effect is implemented, different program logic rules can be derived, enabling reasoning about non-deterministic and/or stochastic GItrees. Unfortunately, these two alternative reifications cannot be defined in the current framework since a reifier is a Rocq function, which is always pure and total.

With that said, the implicit state rules, i.e. WP-PRNG-NEW, WP-PRNG-SEED, and WP-PRNG-GEN, are sound for pseudo-random, non-deterministic and probabilistic number generators. These rules require that the program can be executed safely regardless of the number being generated. Programs that are proven to be safe with this set of rules are safe no matter how we choose to interpret the effect. Specifically, the partial correctness proof of Las Vegas program will remain to be sound if we replace the PRNG reifier with a probabilistic sampling reifier.

There are multiple ways to generalize the GItree theory to lift the aforementioned limitation:

1. Define a reifier as a binary relation over $(\mathrm{IT}, \mathit{State})$ with richer information (probability, weight, etc). The resulting theory is more operational in the sense reasoning about GItrees will be carried out in an operational style.

2. Define a reifiers as a monadic function and endow GItrees with trace semantics. This approach likely gives a more denotational theory.

It is currently unclear whether these approaches are feasible.

4.2 Applicative bisimulation for GItrees

We would like to develop a bisimulation for identifying GItrees that have essentially the same behavior but differs in the number of Ticks takes to evaluate them. More formally, we want to define a binary relation $\sim\colon \mathrm{IT}_E(A) \times \mathrm{IT}_E(A)$ validating the following rules:

$$\frac{}{\alpha \sim \alpha} \qquad \frac{\alpha \sim \beta}{\beta \sim \alpha} \qquad \frac{\alpha \sim \beta \qquad \beta \sim \gamma}{\alpha \sim \gamma}$$

$$\frac{\alpha \sim \mathrm{Tick}(\beta)}{\alpha \sim \beta} \qquad \frac{\forall \alpha.\ f\ \alpha \sim g\ \alpha}{\mathrm{Fun}(f) \sim \mathrm{Fun}(g)} \qquad \frac{x \sim y \qquad \forall \beta.\ k\ \beta \sim k'\ \beta}{\mathrm{Vis}_i(x,k) \sim \mathrm{Vis}_i(y,k')}$$

Ideally, we would like to re-develop the binary logical relation for representation independence in Timany et al. [Tim+24] for GItrees. However, this is not possible due to the lack of the *existential property*. Some discussion on this issue can be found in Sergei Stepanenko's PhD thesis [Ste25].

4.3 Full abstraction for GItree-based denotational semantics

The GItree denotational interpretation of λ_{prng} is fairly intensional. We would like to see whether it is fully abstract.

4.4 Comparing GItree and SGDT

Various semantic domains have been developed in intensional Guarded Dependent Type theory (iGDTT). Paviotti et al. [PMB15; MP16] defined computationally adequate denotational semantics for PCF and FPC based on the guarded delay monad. Møgelberg et al. [MV21] defined powerdomains for modeling non-determinism. Stanssen et al. [Sta+25] proposed an interpretation of probabilistic FPC in the domain of guarded convex delay algebras. It would be interesting to compare SGDT-based denotational semantics and GItree-based denotational semantics.

5 Declaration on Generative AI usage

Generative AI is employed in the report write-up phase of this project. I used GAI for proofreading and polishing. Some informal proofs in the report are created by asking GAI to translate mechanized Rocq proofs.

6 Acknowledgement

Special thanks to Sergei Stepanenko for discussion and proofreading.

References

[Tai67] William W Tait. "Intensional interpretations of functionals of finite type I". In: *The journal of symbolic logic* 32.2 (1967), pp. 198–212.

[Gir72] Jean-Yves Girard. "Interpretation Functionelle et Elimination des Coupures Dans L'arithmetique D'ordre Superieure". PhD thesis. Universite Paris VII, Paris, 1972.

[Rey83] John C Reynolds. "Types, abstraction and parametric polymorphism". In: *Information Processing 83, Proceedings of the IFIP 9th World Computer Congres*. 1983, pp. 513–523.

[CG90] Thierry Coquand and Jean H Gallier. "A proof of strong normalization for the theory of constructions using a Kripke-like interpretation". In: (1990). URL: https://www.seas.upenn.edu/~cis5110/sntoc-new.pdf.

[Mog91] Eugenio Moggi. "Notions of computation and monads". In: *Inf. Comput.* 93.1 (July 1991), pp. 55–92. ISSN: 0890-5401. DOI: 10.1016/0890-5401(91)90052-4.

[OT93] Peter W O'Hearn and Robert D Tennent. "Relational parametricity and local variables". In: *POPL*. 1993, pp. 171–184. URL: https://www.lfcs.inf.ed.ac.uk/reports/92/ECS-LFCS-92-223/index.html.

[Jun+96] Achim Jung et al. "Domains and denotational semantics: History, accomplishments and open problems". In: *School of Computer Science Research Reports-University of BIRMINGHAM CSR* (1996).

[Pit96] Andrew M Pitts. "Reasoning about local variables with operationally-based logical relations". In: *Proceedings 11th Annual IEEE Symposium on Logic in Computer Science*. IEEE. 1996, pp. 152–163.

[Nak00] H. Nakano. "A modality for recursion". In: *Proceedings Fifteenth Annual IEEE Symposium on Logic in Computer Science (Cat. No.99CB36332)*. 2000, pp. 255–266. DOI: 10.1109/LICS.2000.855774.

[AM01] Andrew W. Appel and David McAllester. "An indexed model of recursive types for foundational proof-carrying code". In: *ACM Trans. Program. Lang. Syst.* 23.5 (Sept. 2001), pp. 657–683. ISSN: 0164-0925. DOI: 10.1145/504709.504712.

[Ahm04] Amal Jamil Ahmed. "Semantics of types for mutable state". PhD thesis. Princeton University, 2004. URL: https://www.ccs.neu.edu/home/amal/ahmedsthesis.pdf.

[Cap05] Venanzio Capretta. "General Recursion via Coinductive Types". In: *Logical Methods in Computer Science* Volume 1, Issue 2, 6 (July 2005). ISSN: 1860-5974. DOI: 10.2168/LMCS-1(2:1)2005.

[Bir+12] Lars Birkedal et al. "First steps in synthetic guarded domain theory: step-indexing in the topos of trees". In: *Logical Methods in Computer Science* 8 (2012). URL: https://cs.au.dk/~birke/papers/sgdt-journal.pdf.

[BM13] Lars Birkedal and Rasmus Ejlers Møgelberg. "Intensional type theory with guarded recursive types qua fixed points on universes". In: *2013 28th Annual ACM/IEEE Symposium on Logic in Computer Science*. IEEE. 2013, pp. 213–222. URL: https://cs.au.dk/~birke/papers/sgdtuniverse-conf.pdf.

[PG14] Maciej Piróg and Jeremy Gibbons. "The Coinductive Resumption Monad". In: *Electronic Notes in Theoretical Computer Science* 308 (2014). Proceedings of the 30th Conference on the Mathematical Foundations of Programming Semantics (MFPS XXX), pp. 273–288. ISSN: 1571-0661. DOI: https://doi.org/10.1016/j.entcs.2014.10.015.

[McB15] Conor McBride. "Turing-Completeness Totally Free". In: *Mathematics of Program Construction*. Ed. by Ralf Hinze and Janis Voigtländer. Cham: Springer International Publishing, 2015, pp. 257–275. ISBN: 978-3-319-19797-5.

[PMB15] Marco Paviotti, Rasmus Ejlers Møgelberg, and Lars Birkedal. "A model of PCF in guarded type theory". In: *Electronic Notes in Theoretical Computer Science* 319 (2015), pp. 333–349.

[Biz+16] Aleš Bizjak et al. "Guarded dependent type theory with coinductive types". In: *International Conference on Foundations of Software Science and Computation Structures*. Springer. 2016, pp. 20–35.

[MP16] Rasmus Ejlers Møgelberg and Marco Paviotti. "Denotational semantics of recursive types in synthetic guarded domain theory". In: *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science*. 2016, pp. 317–326.

[KTB17] Robbert Krebbers, Amin Timany, and Lars Birkedal. "Interactive proofs in higher-order concurrent separation logic". In: *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*. 2017, pp. 205–217.

[Jun+18] Ralf Jung et al. "Iris from the ground up: A modular foundation for higher-order concurrent separation logic". In: *Journal of Functional Programming* 28 (2018), e20. DOI: 10.1017/S0956796818000151.

[Kre+18] Robbert Krebbers et al. "MoSeL: a general, extensible modal framework for interactive proofs in separation logic". In: *Proc. ACM Program. Lang.* 2.ICFP (July 2018). DOI: 10.1145/3236772.

[Bir+19] Lars Birkedal et al. "Guarded cubical type theory". In: *Journal of Automated Reasoning* 63.2 (2019), pp. 211–253.

[Sko19] Lau Skorstengaard. *An Introduction to Logical Relations*. 2019. URL: http://arxiv.org/abs/1907.11133.

[Xia+19] Li-yao Xia et al. "Interaction trees: representing recursive and impure programs in Coq". In: *Proc. ACM Program. Lang.* 4.POPL (Dec. 2019). DOI: 10.1145/3371119.

[Car21] Felice Cardone. "Games, Full Abstraction and Full Completeness". In: *The Stanford Encyclopedia of Philosophy*. Ed. by Edward N. Zalta. Spring 2021. Metaphysics Research Lab, Stanford University, 2021. URL: https://plato.stanford.edu/archives/spr2021/entries/games-abstraction/.

[MV21] Rasmus Ejlers Møgelberg and Andrea Vezzosi. "Two guarded recursive powerdomains for applicative simulation". In: *arXiv preprint arXiv:2112.14056* (2021).

[BB23] Lars Birkedal and Aleš Bizjak. *Lecture Notes on Iris: Higher-Order Concurrent Separation Logic*. Aug. 2023. URL: https://iris-project.org/tutorial-pdfs/iris-lecture-notes.pdf.

[FTB24] Dan Frumin, Amin Timany, and Lars Birkedal. "Modular Denotational Semantics for Effects with Guarded Interaction Trees". In: *Proc. ACM Program. Lang.* 8.POPL (Jan. 2024). DOI: 10.1145/3632854.

[Len+24] Meven Lennon-Bertrand et al. *Lecture Notes on Denotational Semantics*. Lecture notes for Part II of the Computer Science Tripos. University of Cambridge. 2024. URL: https://www.cl.cam.ac.uk/teaching/2425/DenotSem/source.pdf.

[Tim+24] Amin Timany et al. "A Logical Approach to Type Soundness". In: *J. ACM* 71.6 (Nov. 2024). ISSN: 0004-5411. DOI: 10.1145/3676954.

[Dre+25] Derek Dreyer et al. *Semantics of Type Systems Lecture Notes*. July 2025. URL: https://plv.mpi-sws.org/semantics-course/lecturenotes.pdf.

[Sta+25] Philipp Stassen et al. "Modelling recursion and probabilistic choice in guarded type theory". In: *Proceedings of the ACM on Programming Languages* 9.POPL (2025), pp. 1417–1445.

[Ste25] Sergei Stepanenko. "Formal Reasoning for Modern Programming Languages". PhD thesis. Aarhus University, Denmark, 2025.

[Ste+25] Sergei Stepanenko et al. "Context-Dependent Effects in Guarded Interaction Trees". In: Hamilton, ON, Canada: Springer-Verlag, 2025, pp. 286–313. ISBN: 978-3-031-91120-0. DOI: 10.1007/978-3-031-91121-7_12.

[VSJ25] Max Vistrup, Michael Sammler, and Ralf Jung. "Program Logics à la Carte". In: *Proc. ACM Program. Lang.* 9.POPL (Jan. 2025). DOI: 10.1145/3704847.

[Dev26] The Iris Deverlopers. *The Iris 4.5 Reference*. May 2026. URL: https://plv.mpi-sws.org/iris/appendix-4.5.pdf.

[Gre+] Simon Gregersen et al. *Iris Tutorial*. URL: https://github.com/logsem/iris-tutorial.